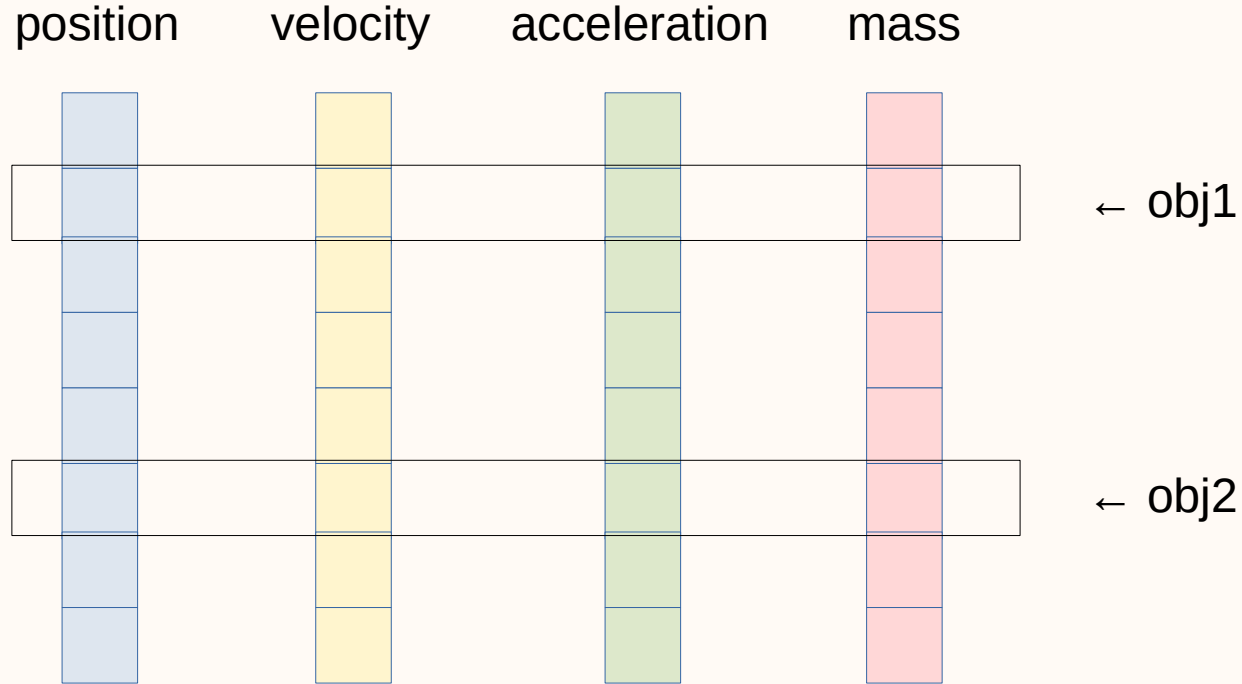


**Cache Friendly Data +
Functional + Ranges = ❤️**

Björn Fahlner

2024

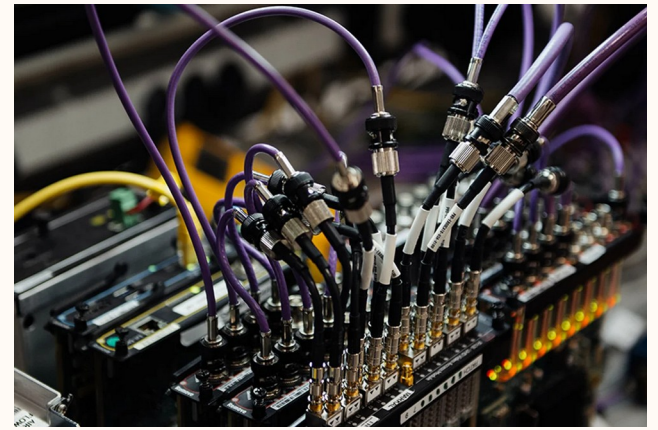
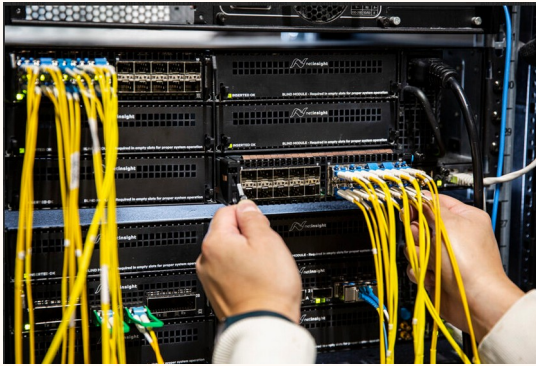
Cache Friendly + Functional + Ranges = ❤️



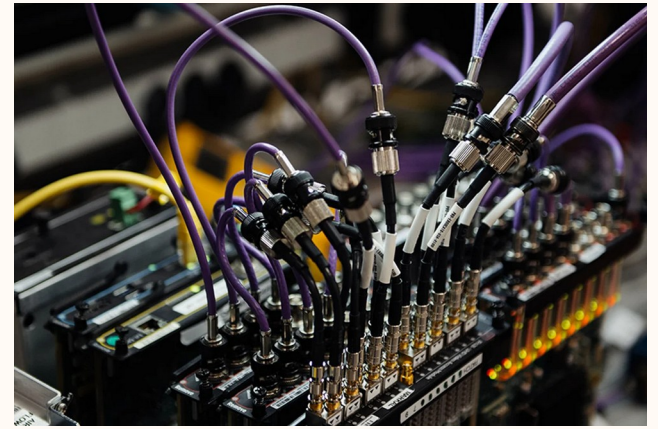
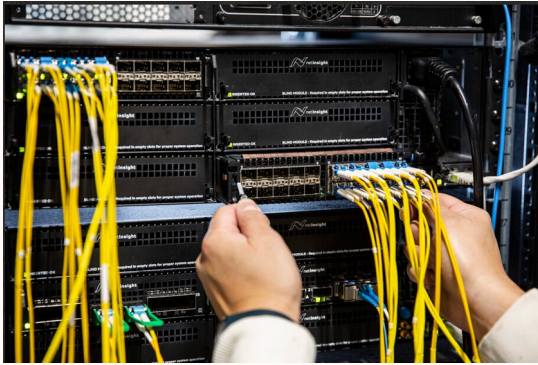
Björn Fahlner



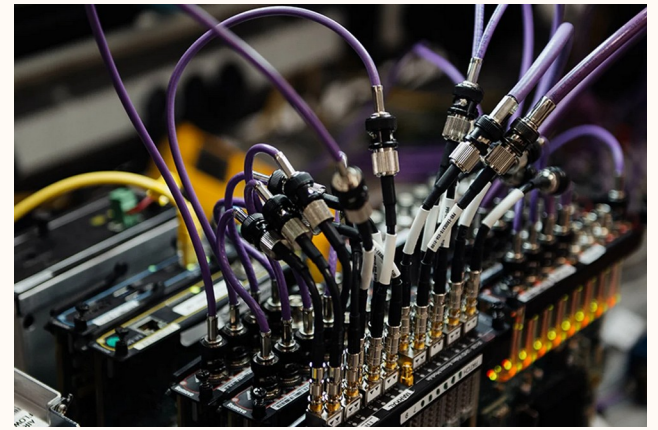
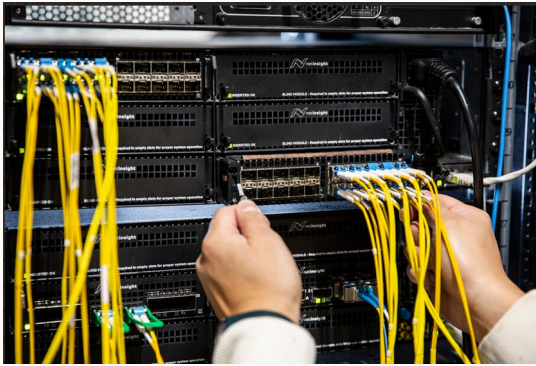
<https://jobs.netinsight.net/departments/development>



<https://jobs.netinsight.net/departments/development>



<https://jobs.netinsight.net/departments/development>

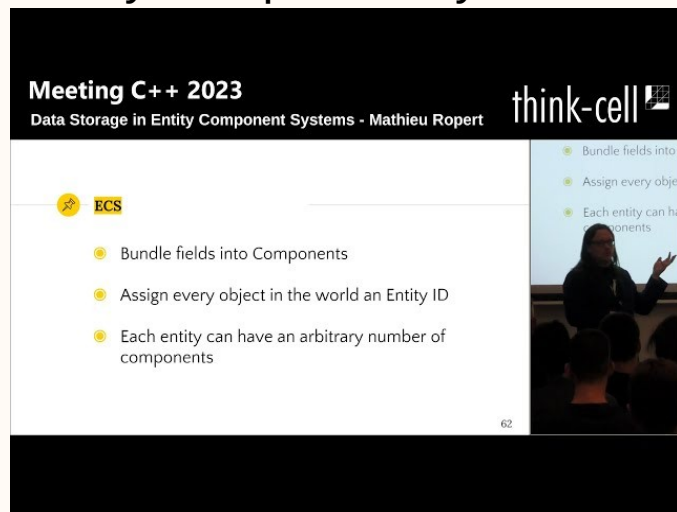


<https://jobs.netinsight.net/departments/development>

Inspiration

Mathieu Ropert
– MeetingC++ 2023

Data Storage in Entity Component System



<https://youtu.be/b9hNKFj5R3Y>

Mike Acton
– CppCon Keynote 2014

Data Oriented Design



<https://youtu.be/rX0ItVEVjHc>

```
struct S { int x; int y; int z; int d };
```

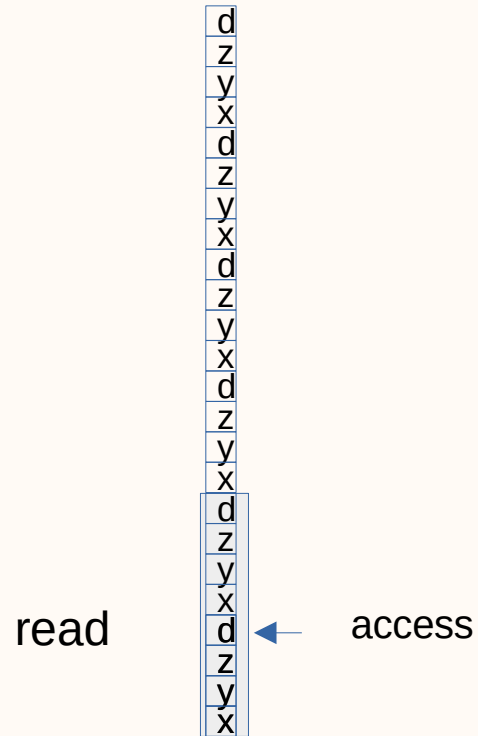
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x

```
struct S { int x; int y; int z; int d };
```

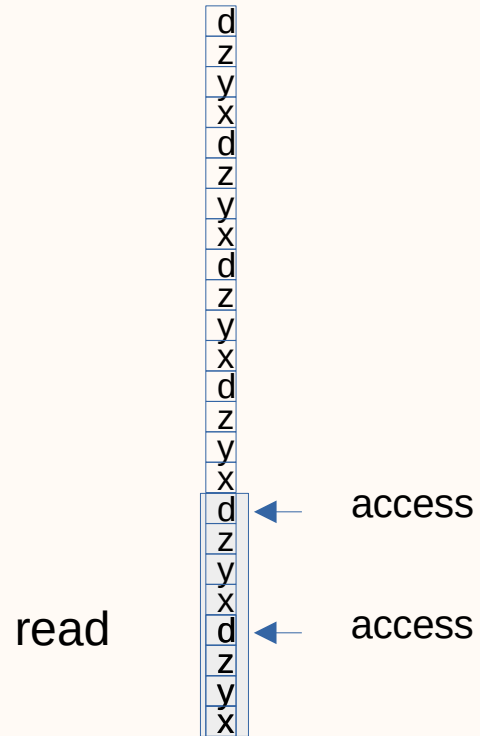
read

d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x

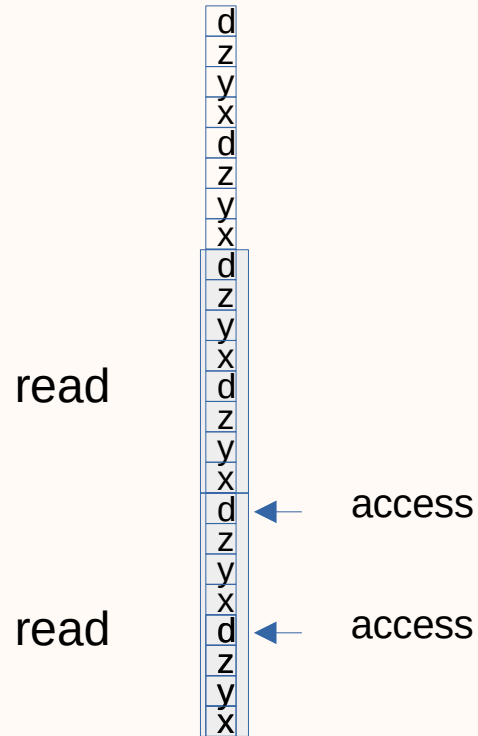
```
struct S { int x; int y; int z; int d };
```



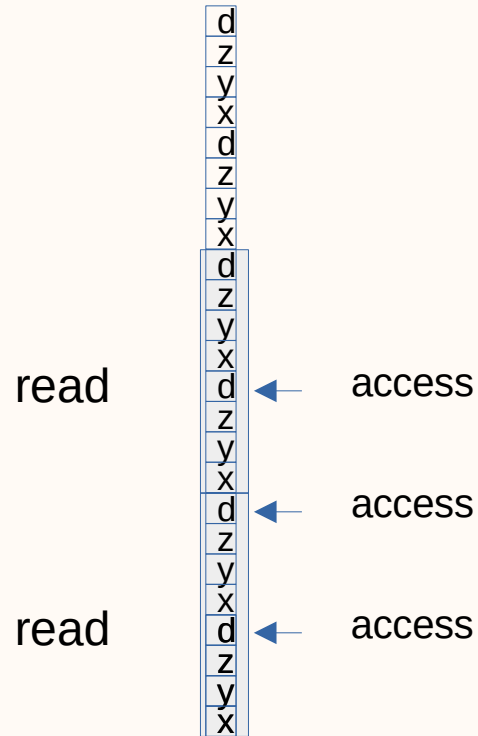
```
struct S { int x; int y; int z; int d };
```



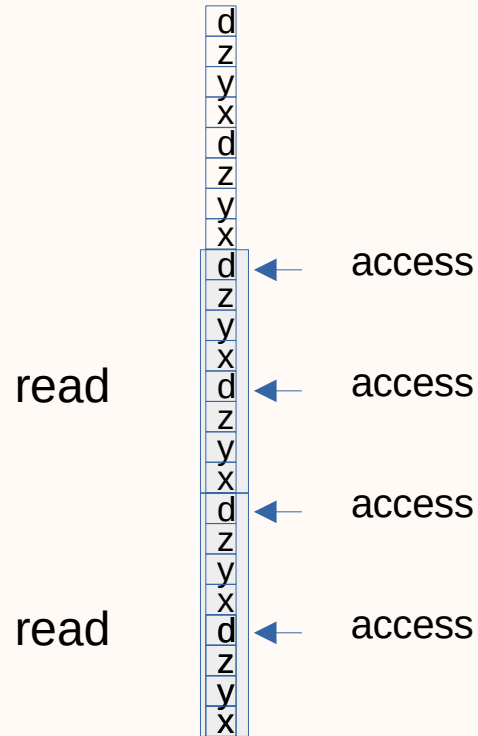
```
struct S { int x; int y; int z; int d };
```



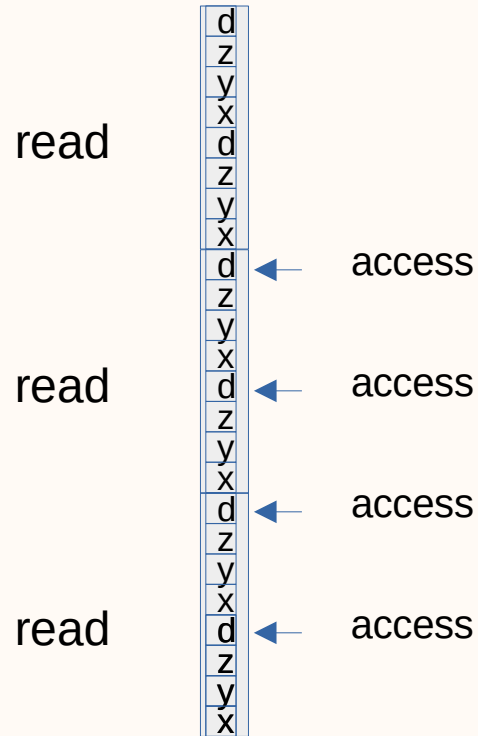
```
struct S { int x; int y; int z; int d };
```



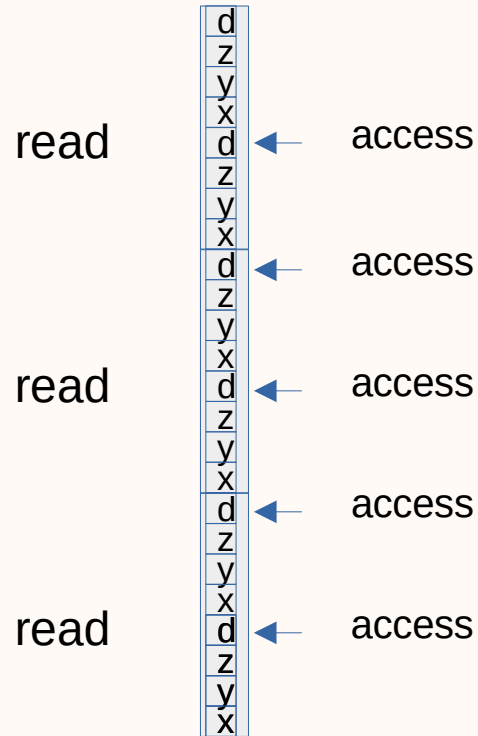
```
struct S { int x; int y; int z; int d };
```



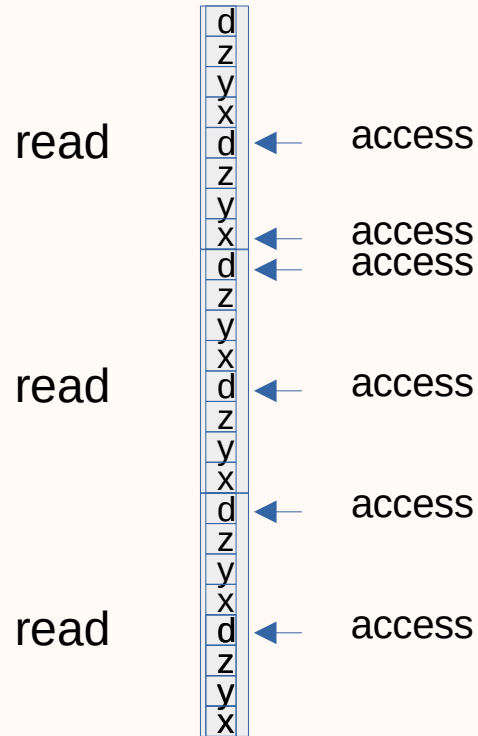
```
struct S { int x; int y; int z; int d };
```



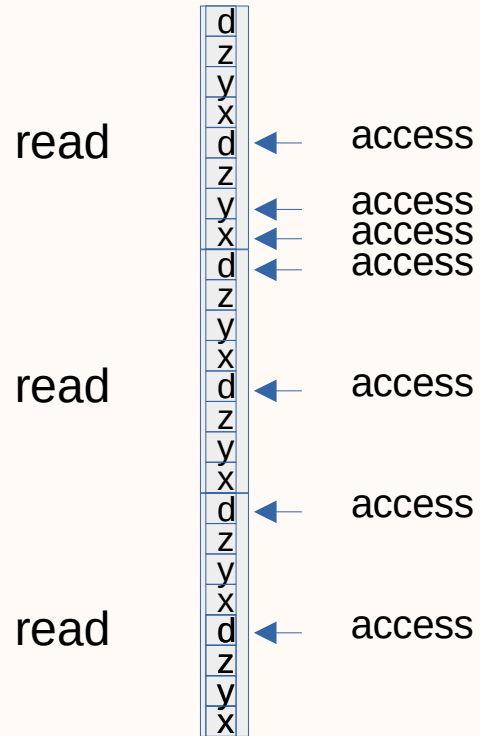
```
struct S { int x; int y; int z; int d };
```



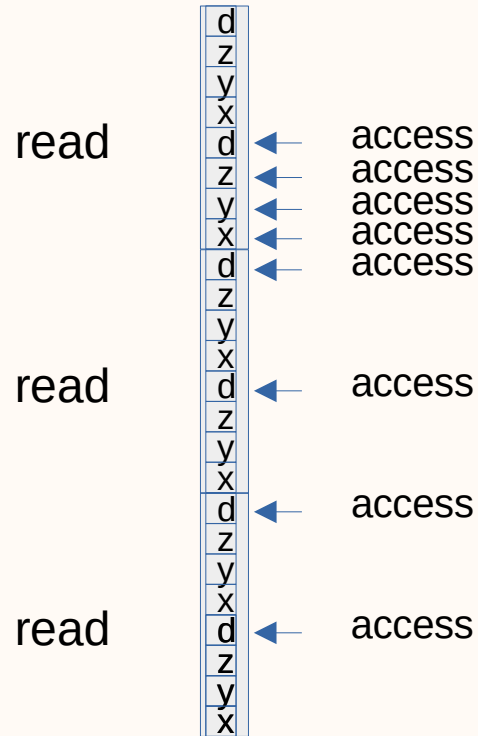
```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```

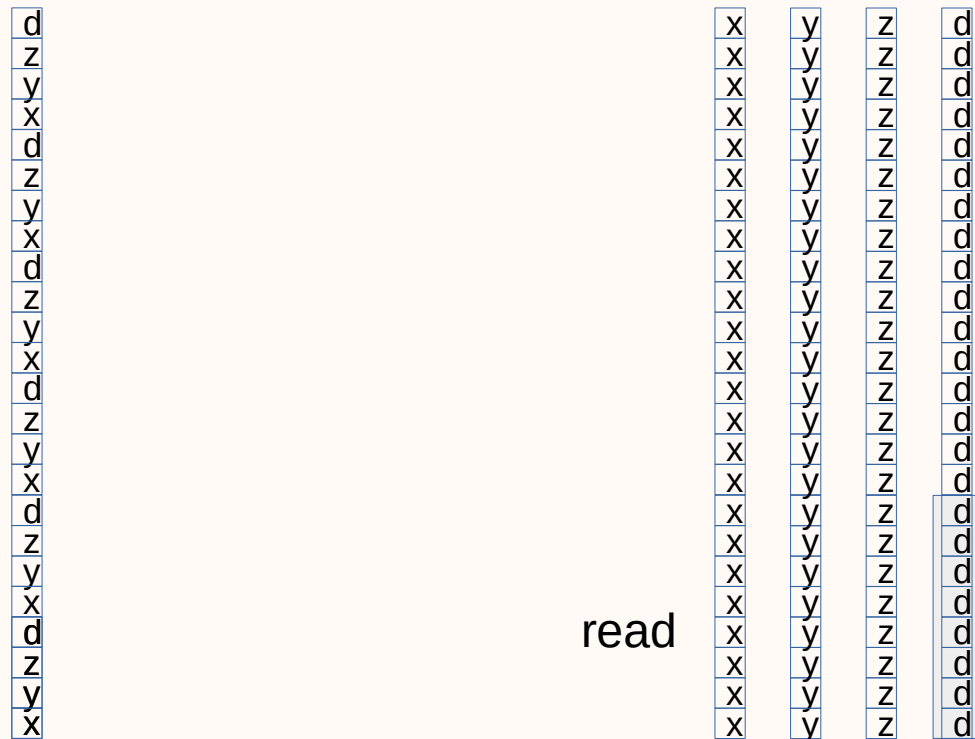


```
struct S { int x; int y; int z; int d };
```

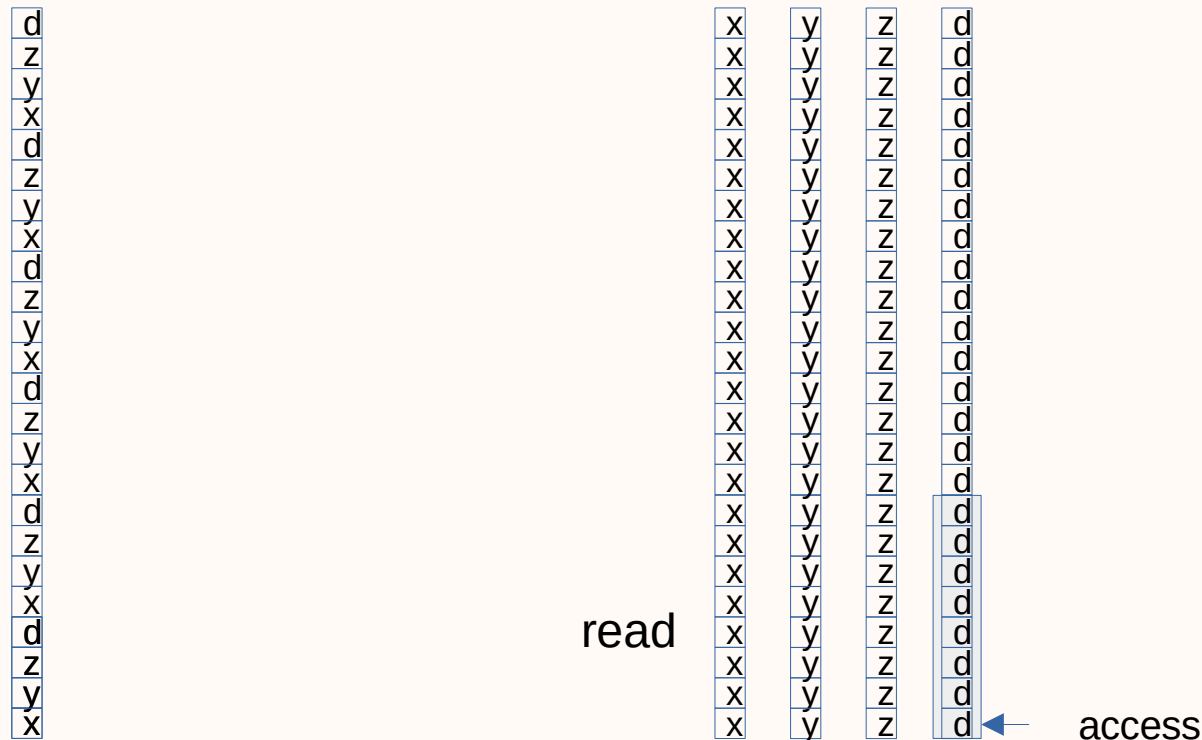
d	z	y	x
z	y	x	d
y	x	d	z
x	d	z	y
d	z	y	x
z	y	x	d
y	x	d	z
x	d	z	y
d	z	y	x
z	y	x	d
y	x	d	z
x	d	z	y

[illegible]

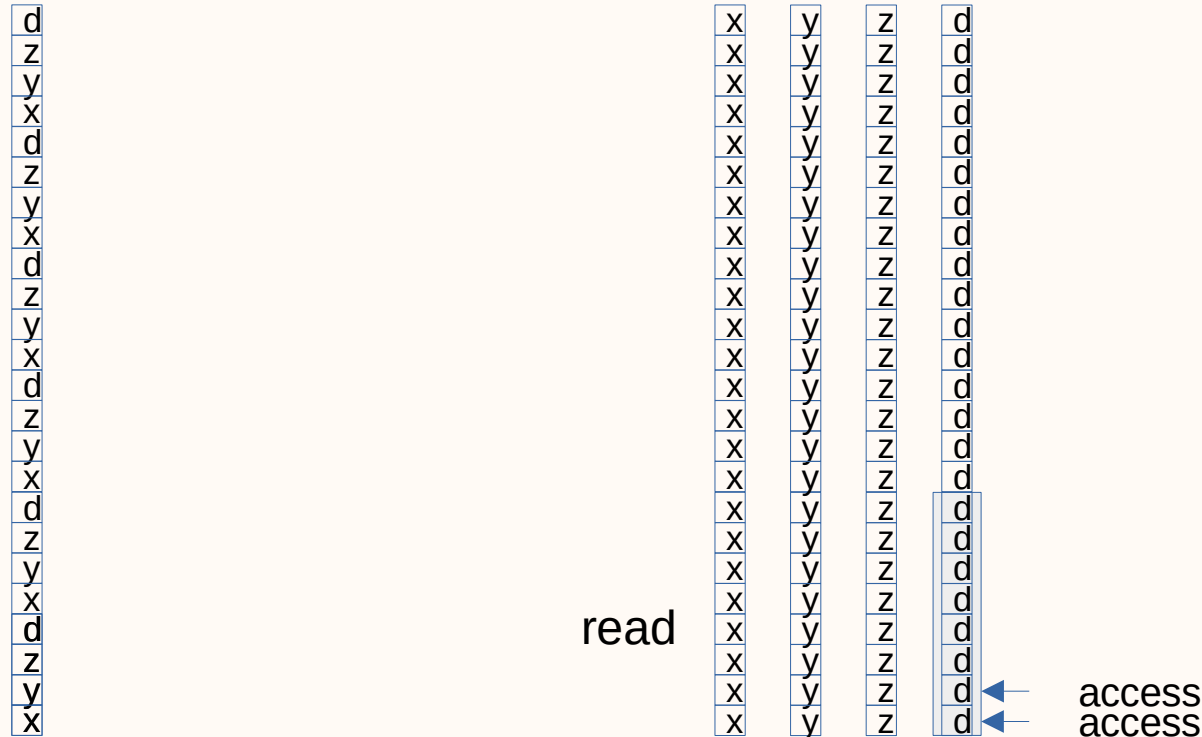
```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```



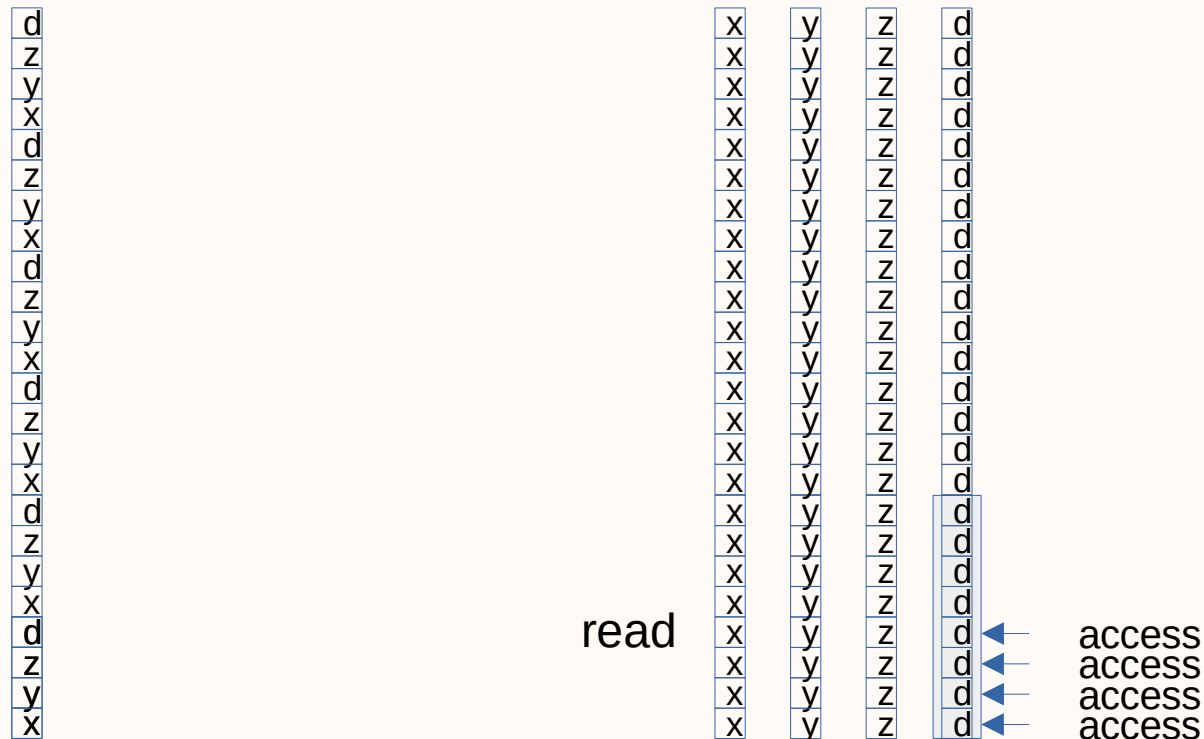
```
struct S { int x; int y; int z; int d };
```



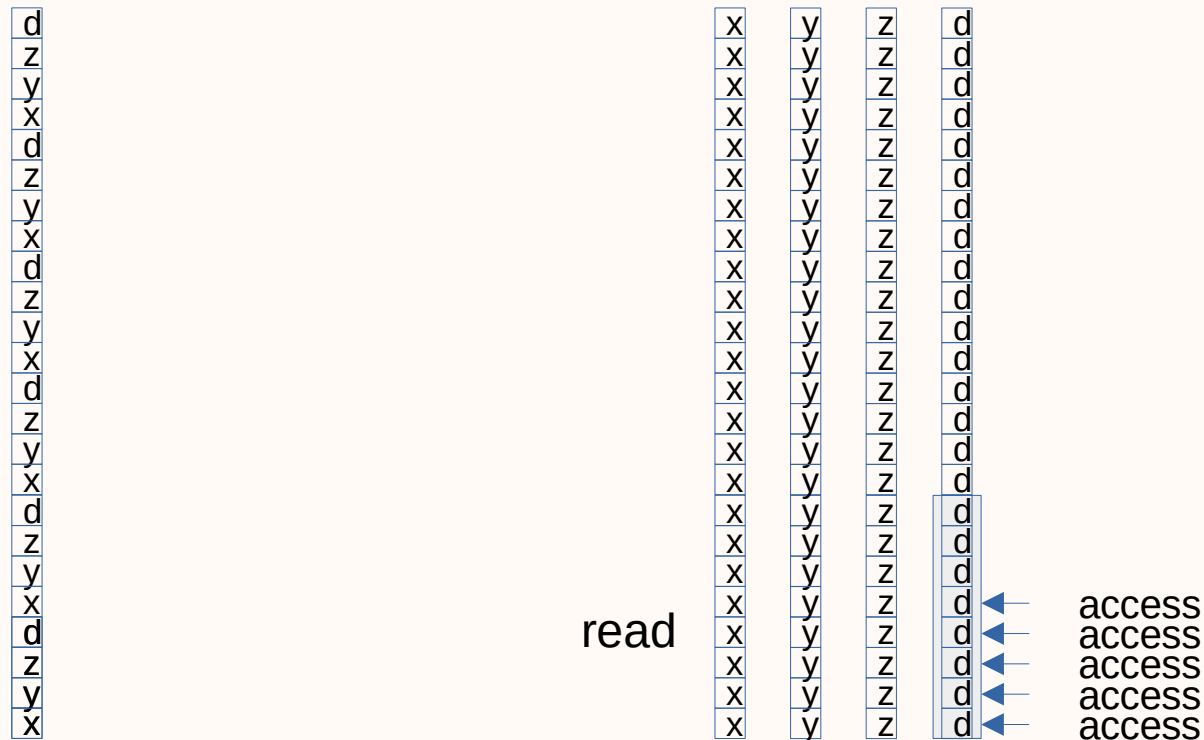
[illegible][illegible][illegible][illegible][illegible]

access
access
access

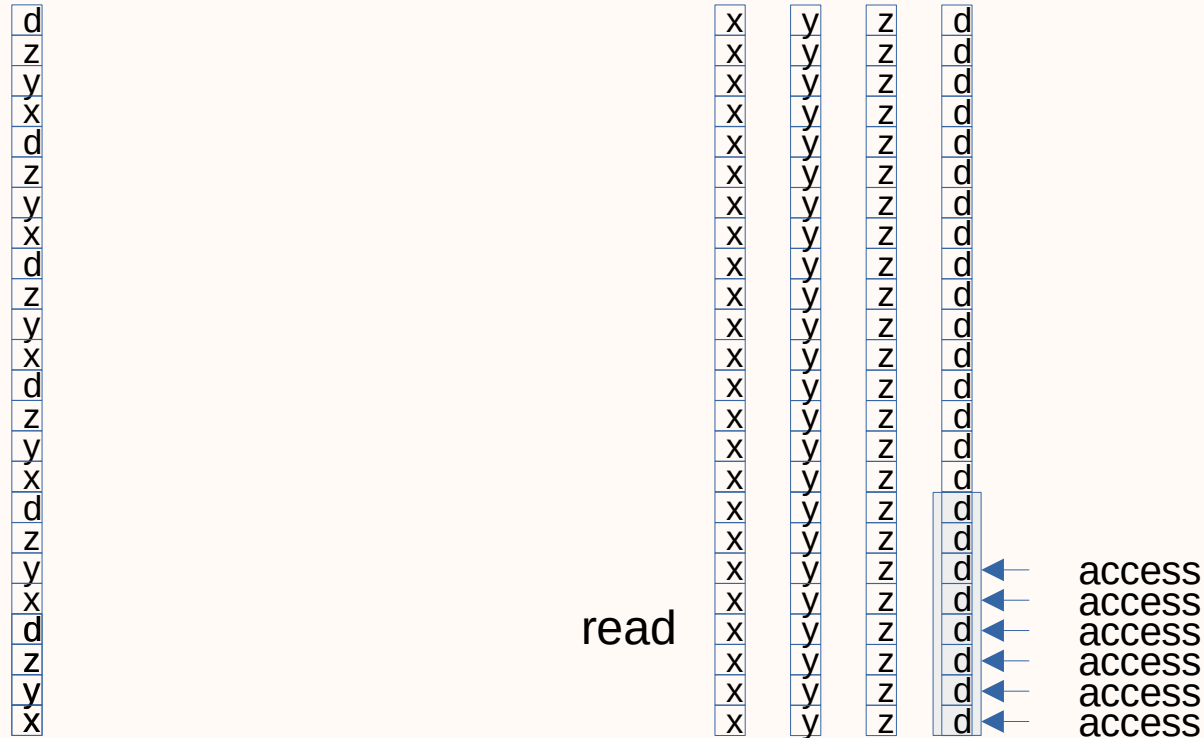
```
struct S { int x; int y; int z; int d };
```



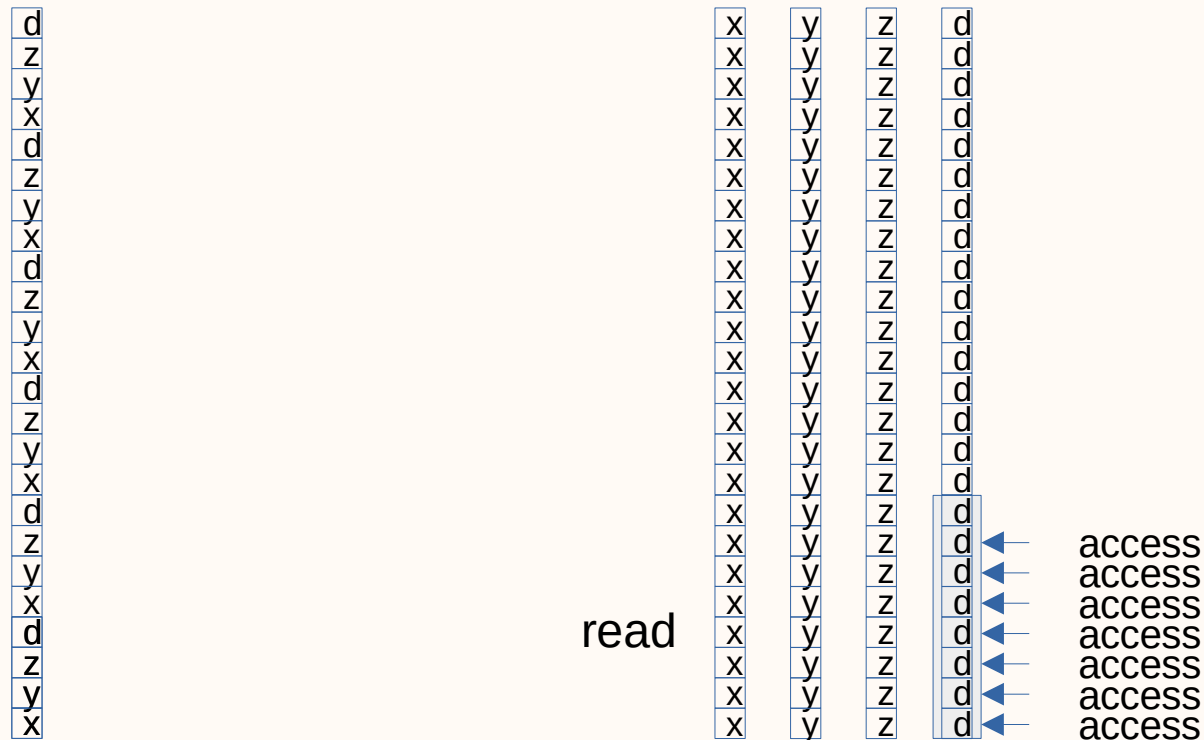
```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```



vector of structs / structs of vectors

```
struct S { int x; int y; int z; int d };
```

d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x

read

x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x

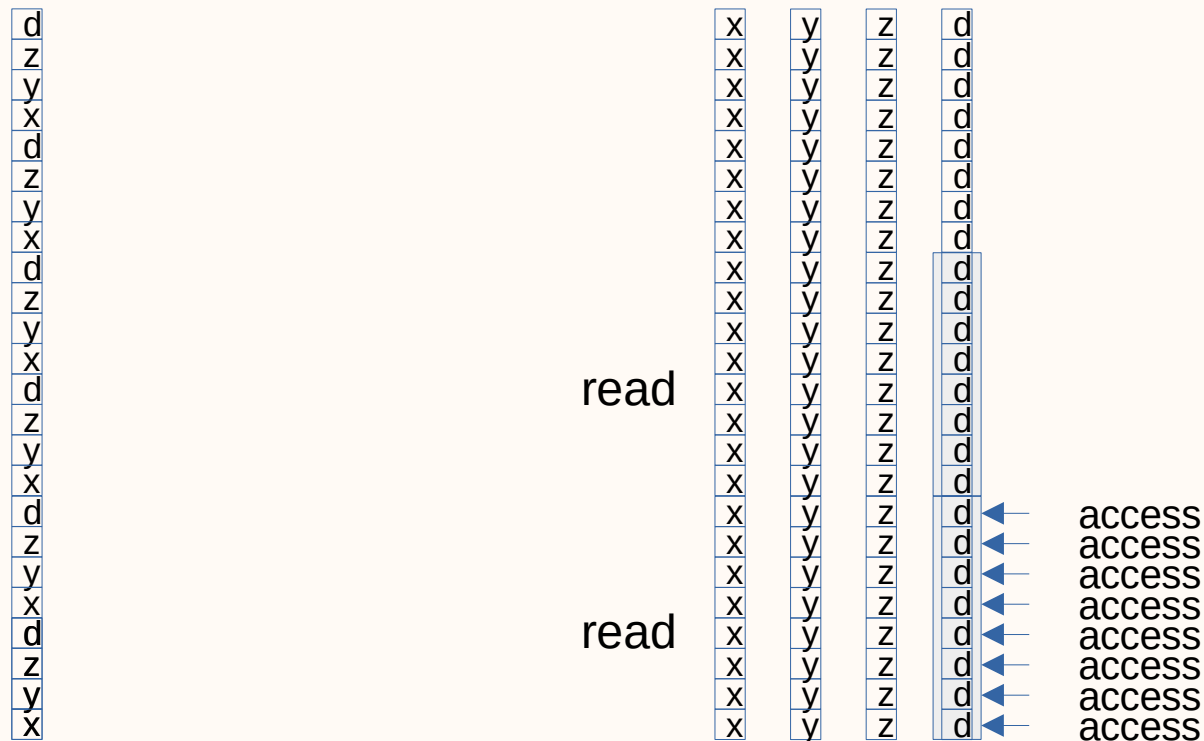
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y

z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z

d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d

access
access
access
access
access
access
access
access
access
access

```
struct S { int x; int y; int z; int d };
```



vector of structs / structs of vectors

```
struct S { int x; int y; int z; int d };
```

d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x

read

read

x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x

y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y

z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z

d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d



access
access
access
access
access
access
access
access
access
access
access
access
access

vector of structs / structs of vectors

```
struct S { int x; int y; int z; int d };
```

d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x

read

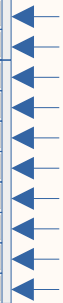
read

x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x

y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y

z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z

d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d



access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access

vector of structs / structs of vectors

```
struct S { int x; int y; int z; int d };
```

d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x
d
z
y
x

read

read

x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x
x

y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y
y

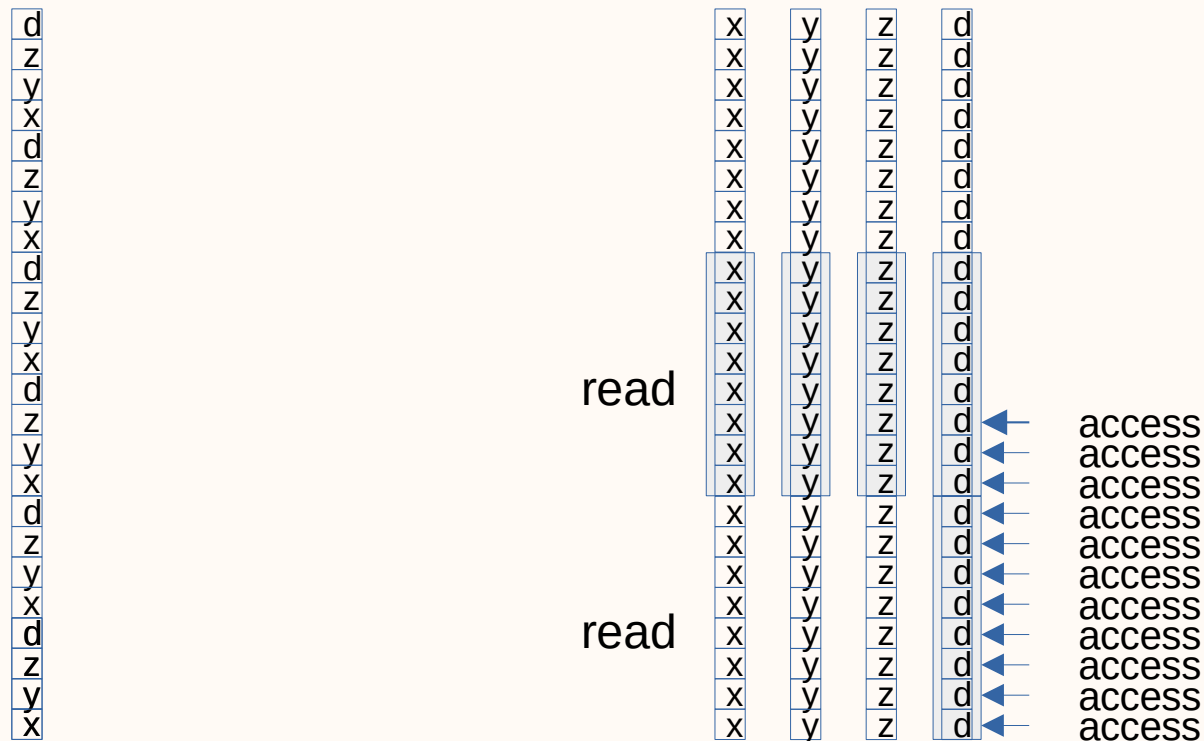
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z
z

d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d
d

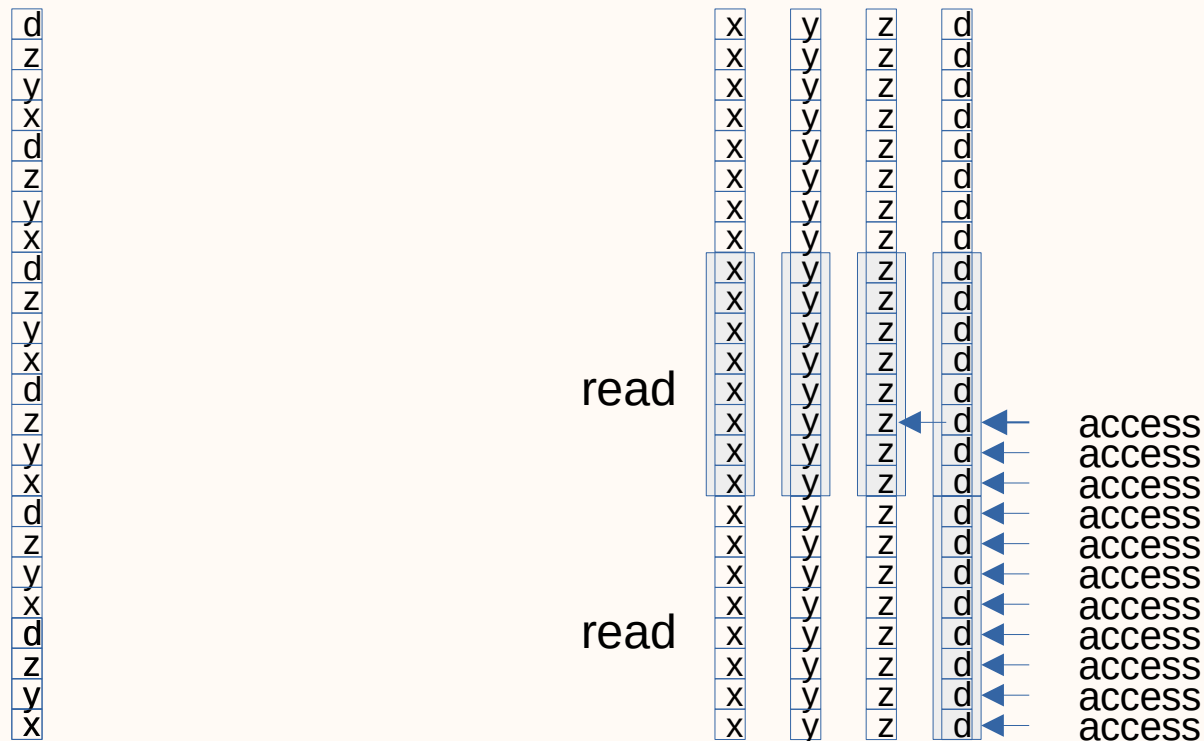


access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access
access

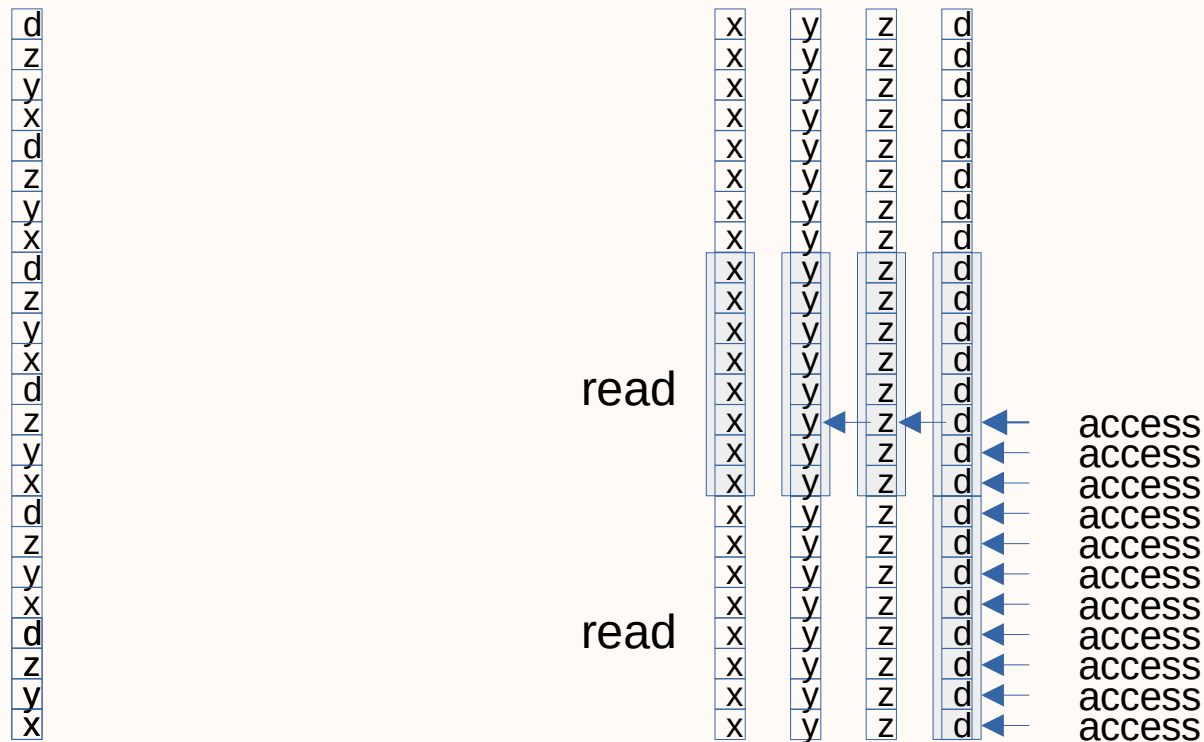
```
struct S { int x; int y; int z; int d };
```



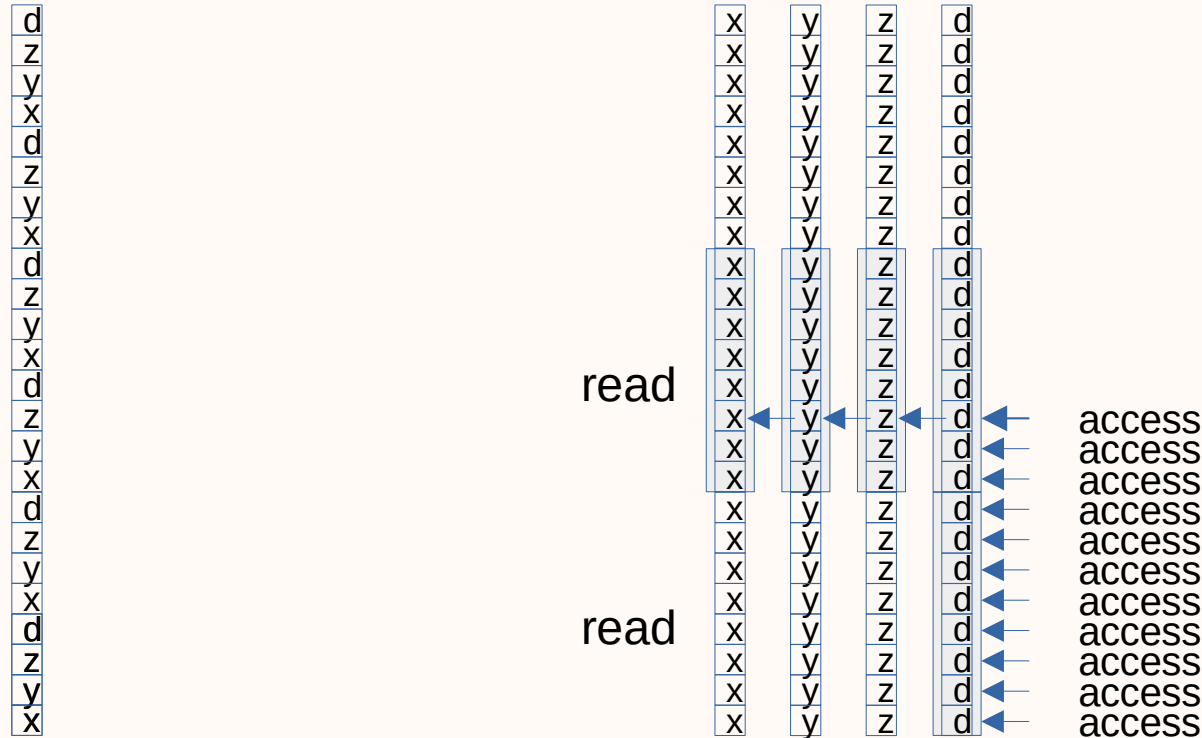
```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```



```
struct S { int x; int y; int z; int d };
```



Live Demo

vector_of_structs vs struct_of_vectors

Observations

In this example, struct of vectors...

Observations

In this example, struct of vectors...

... has way fewer memory accesses

Observations

In this example, struct of vectors...

... has way fewer memory accesses

... has way fewer cache misses

Observations

In this example, struct of vectors...

- ... has way fewer memory accesses
- ... has way fewer cache misses
- ... has slightly fewer instructions

Observations

In this example, struct of vectors...

- ... has way fewer memory accesses
- ... has way fewer cache misses
- ... has slightly fewer instructions
- ... is significantly faster

Observations

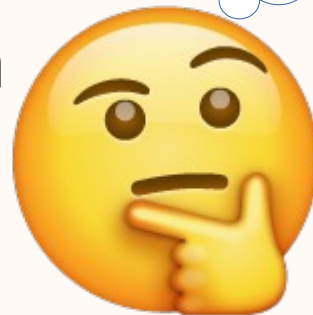
In this example, struct of vectors...

- ... has way fewer memory accesses
- ... has way fewer cache misses
- ... has slightly fewer instructions
- ... is significantly faster
- ... is horrible code to work with

Observations

In this example, struct of vectors...

- ... has way fewer memory accesses
- ... has way fewer cache misses
- ... has slightly fewer instructions
- ... is significantly faster
- ... is horrible code to work with



Can we make it easier to work with and keep the performance?

std::tuple<> is cool!

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    row operator[](size_t i) {
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>)
        {
            return row{std::get<Is>(data_)[i]...};
        };
        return std::invoke(access, indexes);
    }
    ...
private:
    static constexpr std::index_sequence_for<Ts...> indexes{};
    std::tuple<std::vector<Ts>...> data_;
};
```

std::tuple<> is cool!

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    row operator[](size_t i) {
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>)
        {
            return row{std::get<Is>(data_)[i]...};
        };
        return std::invoke(access, indexes);
    }
    ...
private:
    static constexpr std::index_sequence_for<Ts...> indexes{};
    std::tuple<std::vector<Ts>...> data_;
};
```

Storage is a tuple of
vectors of Ts

std::tuple<> is cool!

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    row operator[](size_t i) {
        auto access = [&<size_t i> Ts&...>(std::index_sequence<Is...>)
        {
            return std::get<Is...>(data_[i]...);
        };
        return std::invoke(access, indexes);
    }
    ...
private:
    static constexpr std::index_sequence_for<Ts...> indexes{};
    std::tuple<std::vector<Ts>...> data_;
};
```

Access a row as a tuple of references to the elements

std::tuple<> is cool!

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    row operator[](size_t i) {
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>)
        {
            return row{std::get<Is>(data_)[i]...};
        };
        return std::invoke(access, indexes);
    }
    ...
private:
    static constexpr std::index_sequence_for<Ts...> indexes{};
    std::tuple<std::vector<Ts>...> data_;
};
```

Build by dereferencing
the same offset in each
vector in the tuple

table<Ts...>

table<Ts...>

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;

    void push_back(Ts... ts);
    void pop_back();
    row operator[](size_t i);
    row back();
    void reserve(size_t size);
    bool empty() const;
    size_t size() const
private:
    ...
};
```

table<Ts...>

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;

    void push_back(Ts... ts);
    void pop_back();
    row operator[](size_t i);
    row back();
    void reserve(size_t size);
    bool empty() const;
    size_t size() const
private:
    ...
};
```

```
template <typename Table, typename P>
void drop_if(Table& t, P predicate)
{
    size_t i = 0;
    while (i < t.size()) {
        if (predicate(t[i])) {
            t[i] = std::move(v.back());
            t.pop_back();
        } else {
            ++i;
        }
    }
}
```

table<Ts...>

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;

    void push_back(Ts... ts);
    void pop_back();
    row operator[](size_t i);
    row back();
    void reserve(size_t size);
    bool empty() const;
    size_t size() const;
private:
```

```
template <typename Table, typename P>
void drop_if(Table& t, P predicate)
{
    size_t i = 0;
    while (i < t.size()) {
        if (predicate(t[i])) {
            t[i] = std::move(v.back());
            t.pop_back();
        } else {
            ++i;
        }
    }
}
```

```
while (!values.empty()) {
    drop_if(values, [&](auto r) { auto [x,y,z,d] = r; return z < start; });
    ++iter;
    start += 10;
}
}
```

Live Demo

templated

Observations

In this example...



Observations

In this example...

... the ergonomics is roughly the same as with vector of structs



Observations

In this example...

- ... the ergonomics is roughly the same as with vector of structs
- ... performance is the same as with struct of vectors



Observations

In this example...

- ... the ergonomics is roughly the same as with vector of structs
- ... performance is the same as with struct of vectors



But, we also need...

Observations

In this example...

... the ergonomics is roughly the same as with vector of structs
... performance is the same as with struct of vectors



But, we also need...

... a stable row-ID to lookup objects

Observations

In this example...

- ... the ergonomics is roughly the same as with vector of structs
- ... performance is the same as with struct of vectors



But, we also need...

- ... a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

Observations

In this example...

- ... the ergonomics is roughly the same as with vector of structs
- ... performance is the same as with struct of vectors



But, we also need...

- ... a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

And want...

Observations

In this example...

- ... the ergonomics is roughly the same as with vector of structs
- ... performance is the same as with struct of vectors



But, we also need...

- ... a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

And want...

- ... a convenient way to look at only some columns

Observations

In this example...

- ... the ergonomics is roughly the same as with vector of structs
- ... performance is the same as with struct of vectors



But, we also need...

- ... a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

And want...

- ... a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements

Stable row ID

8	i	I
7	h	H
6	g	G
5	f	F
4	e	E
3	d	D
2	c	C
1	b	B
0	a	A



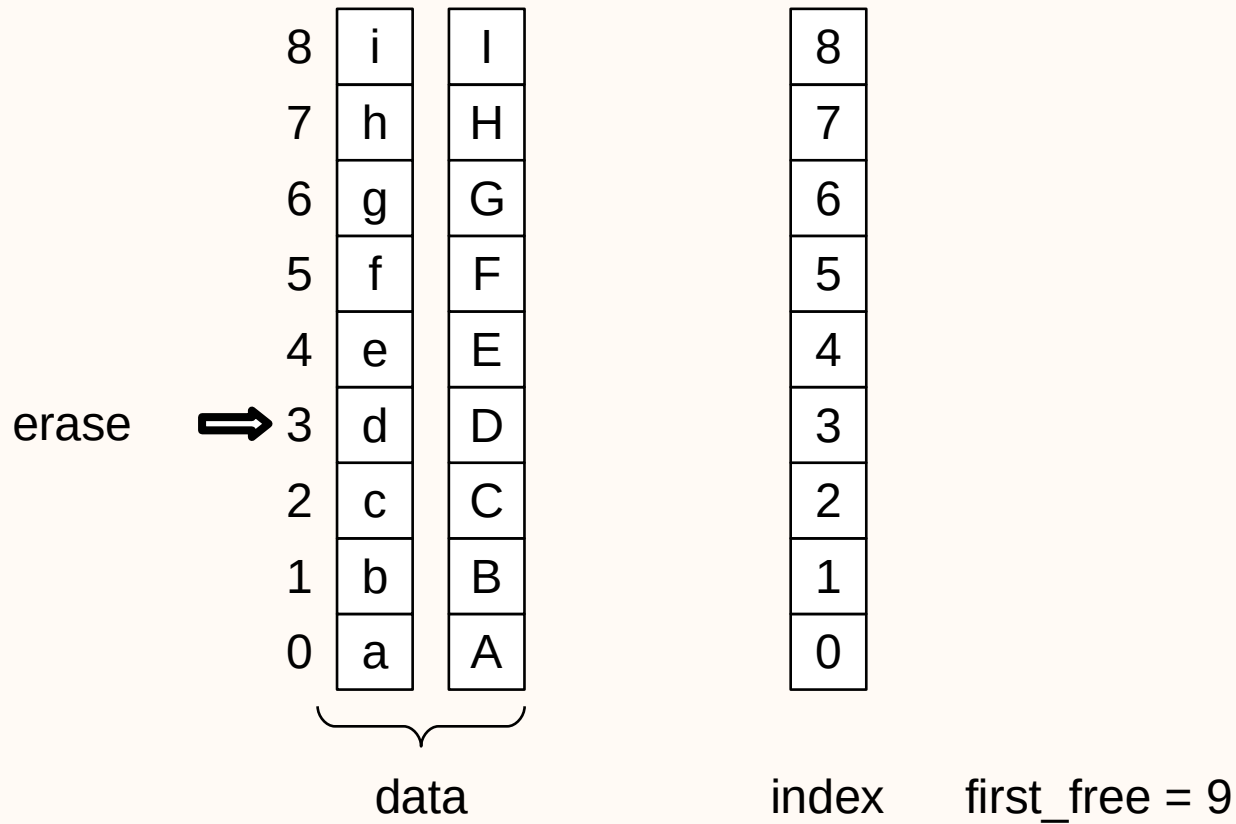
data

8
7
6
5
4
3
2
1
0

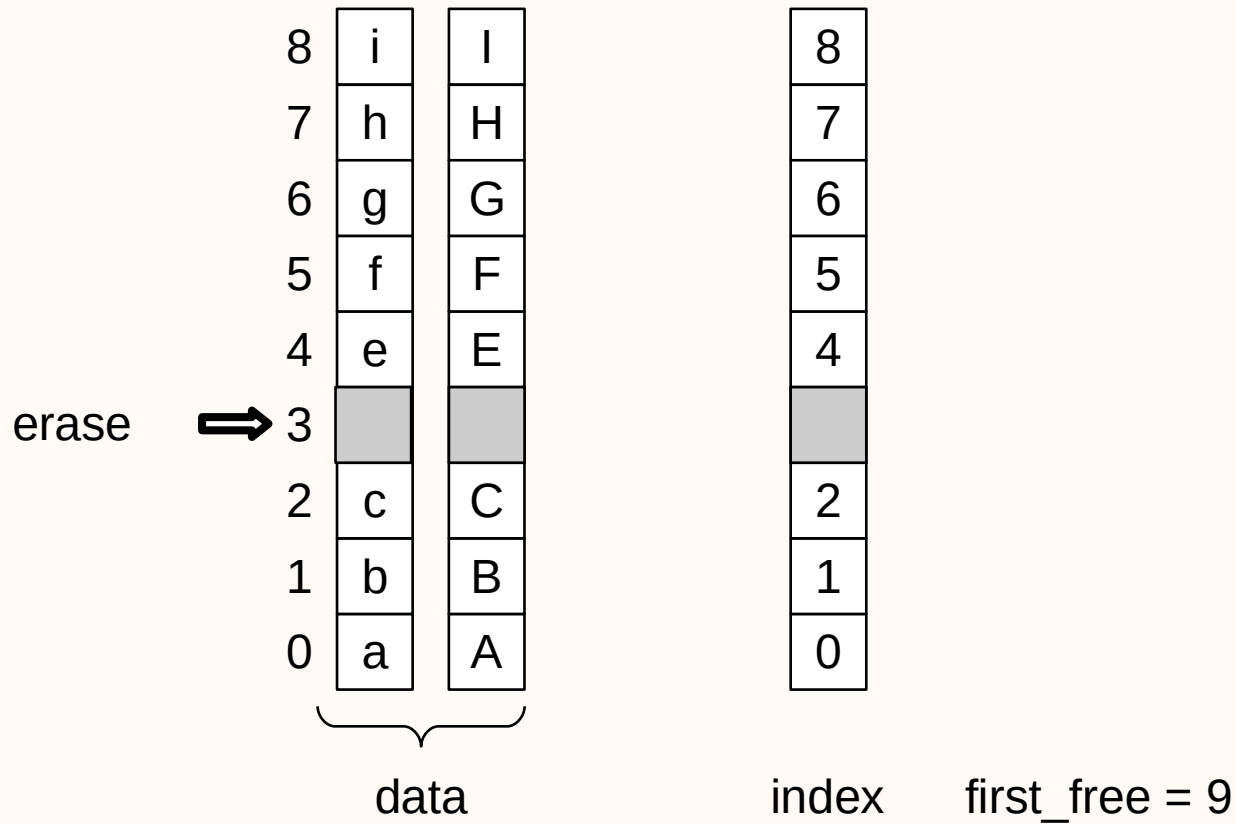
index

first_free = 9

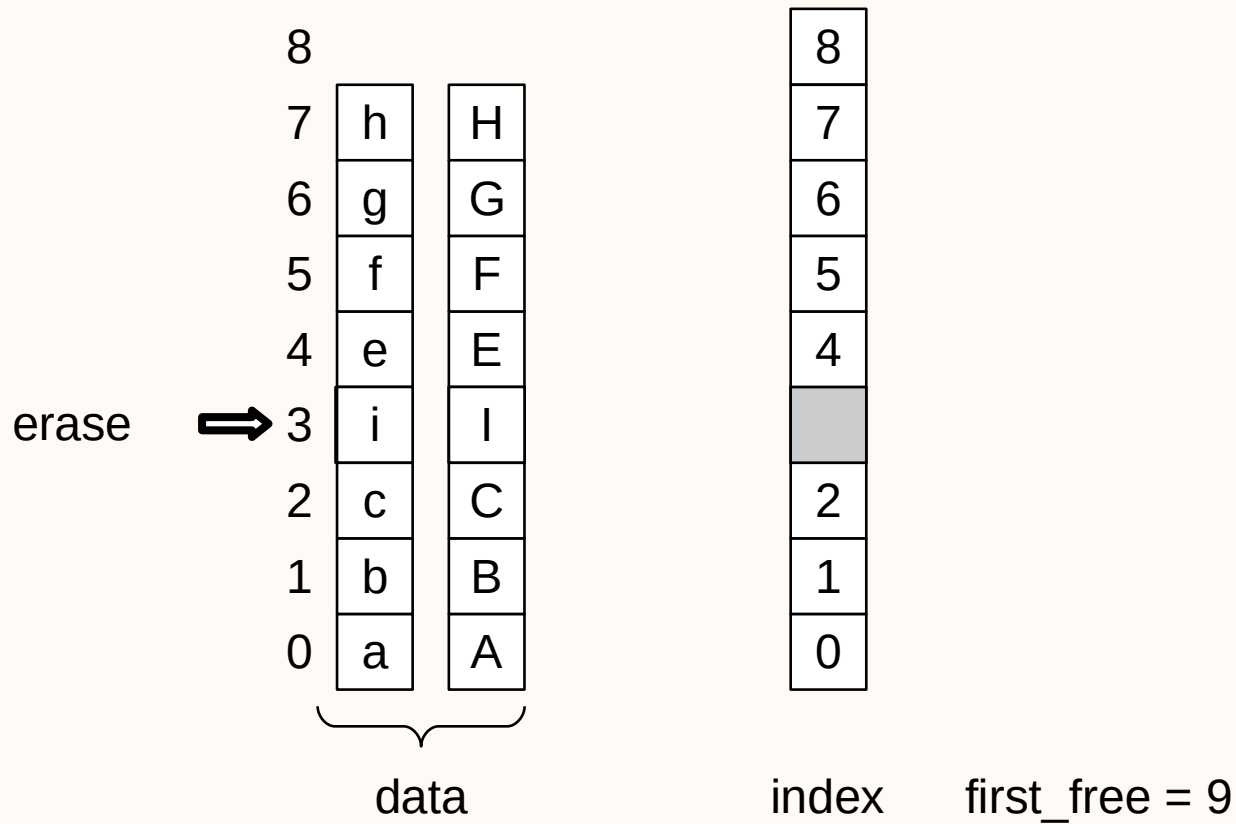
Stable row ID



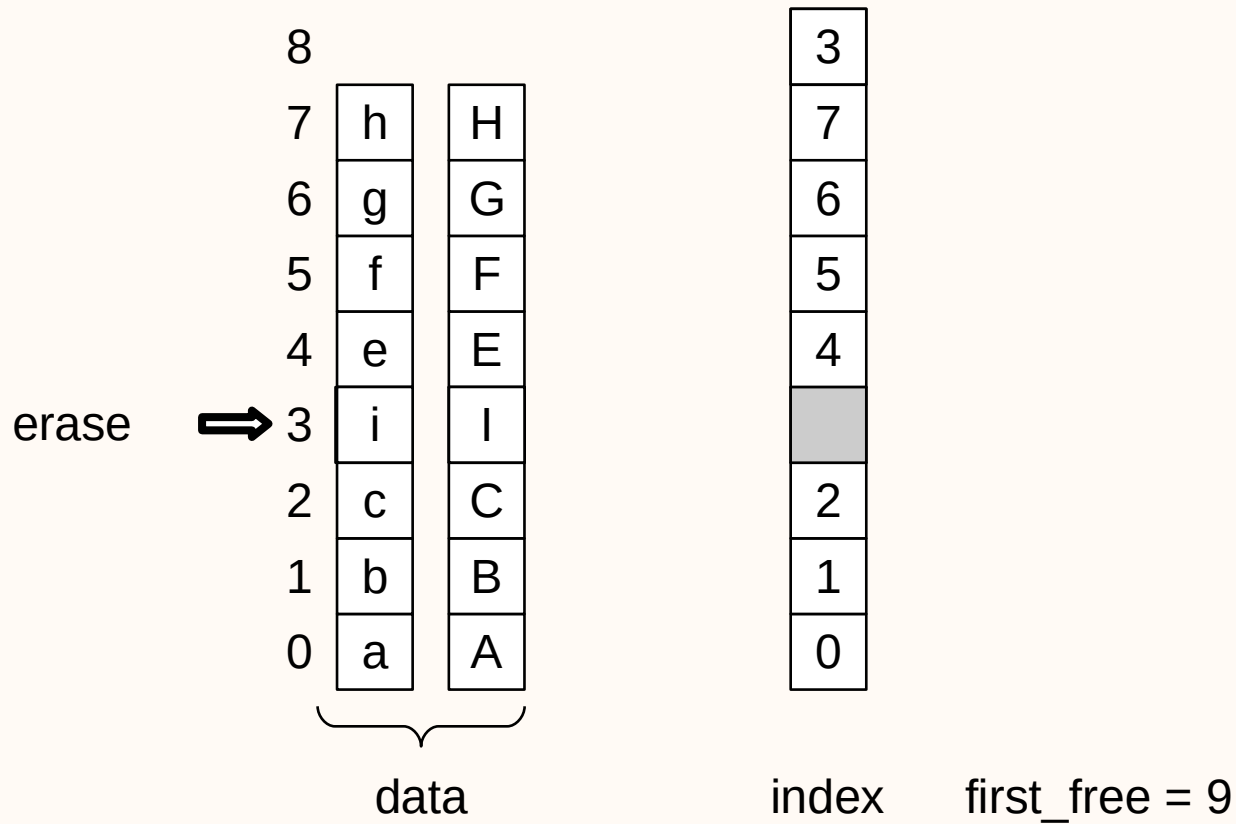
Stable row ID



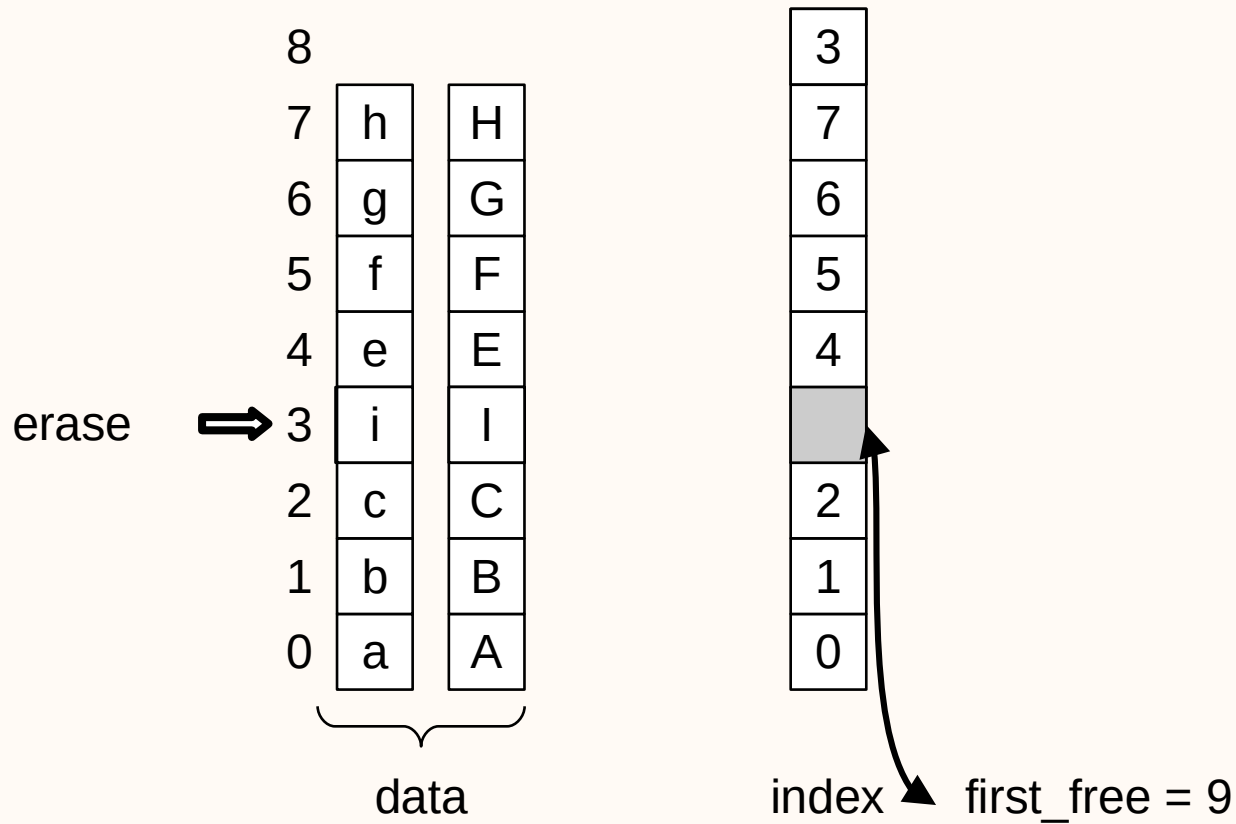
Stable row ID



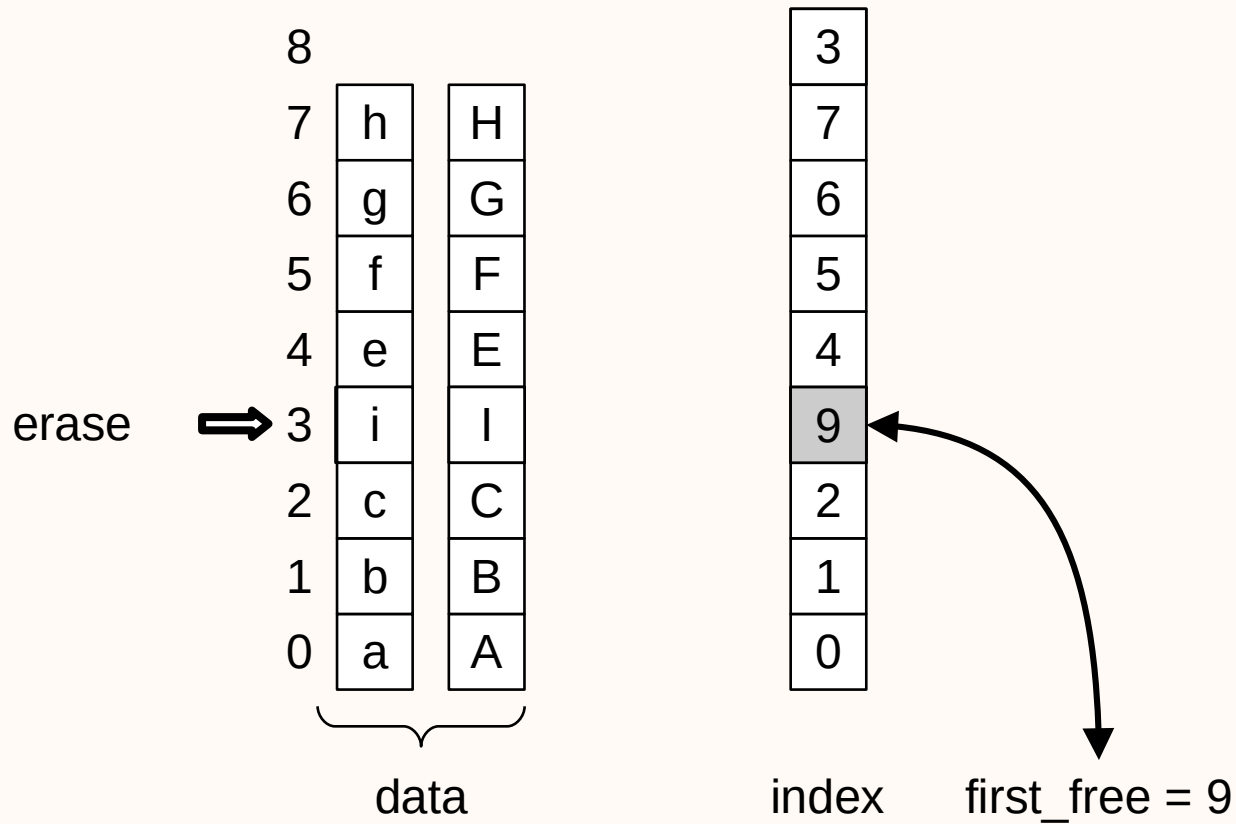
Stable row ID



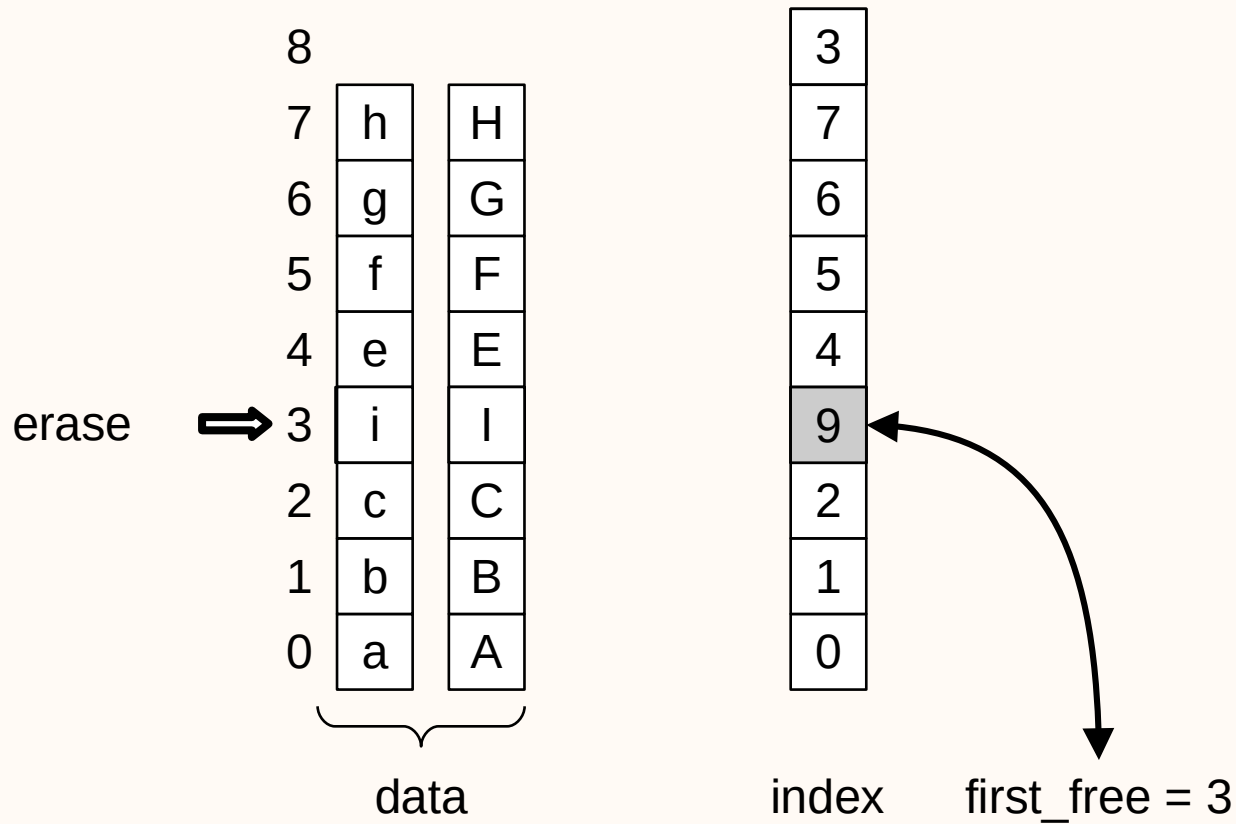
Stable row ID



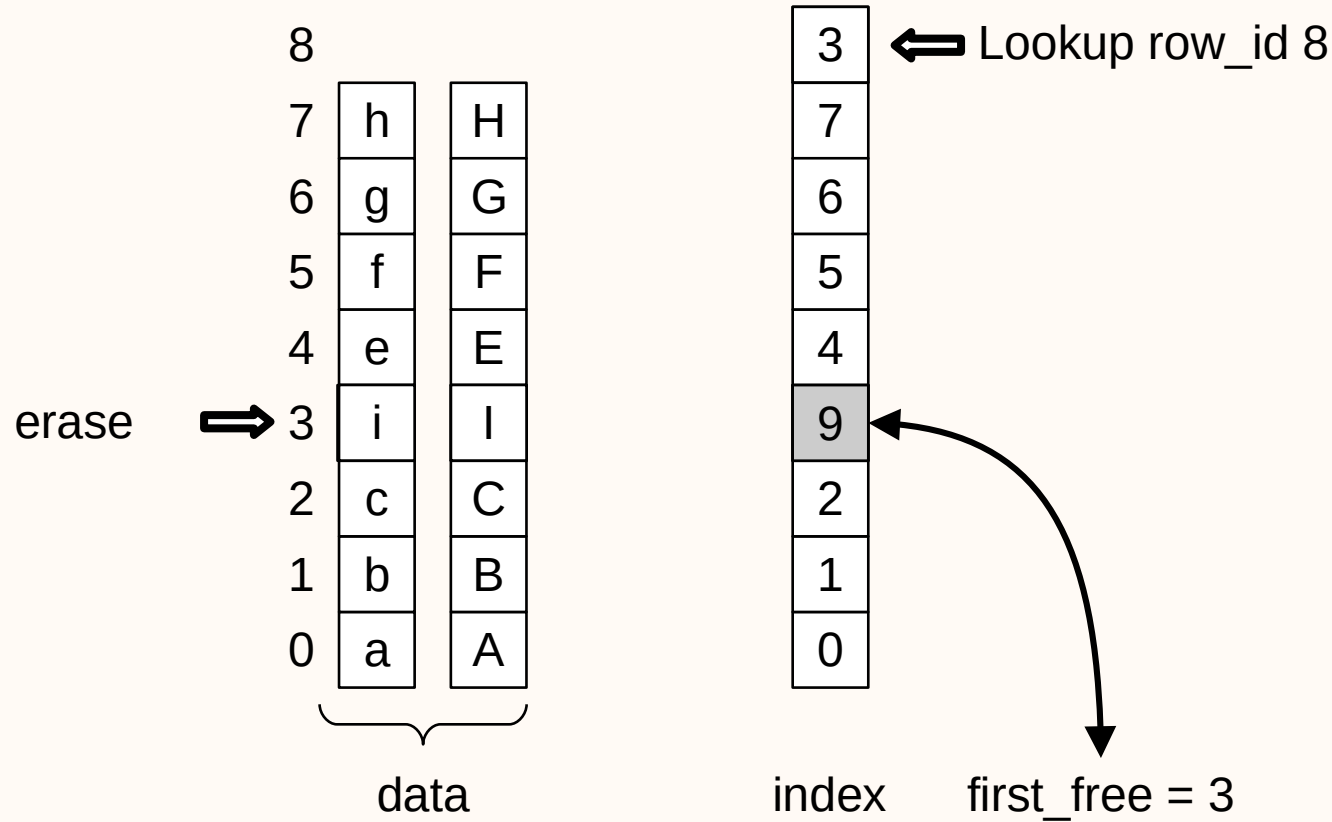
Stable row ID



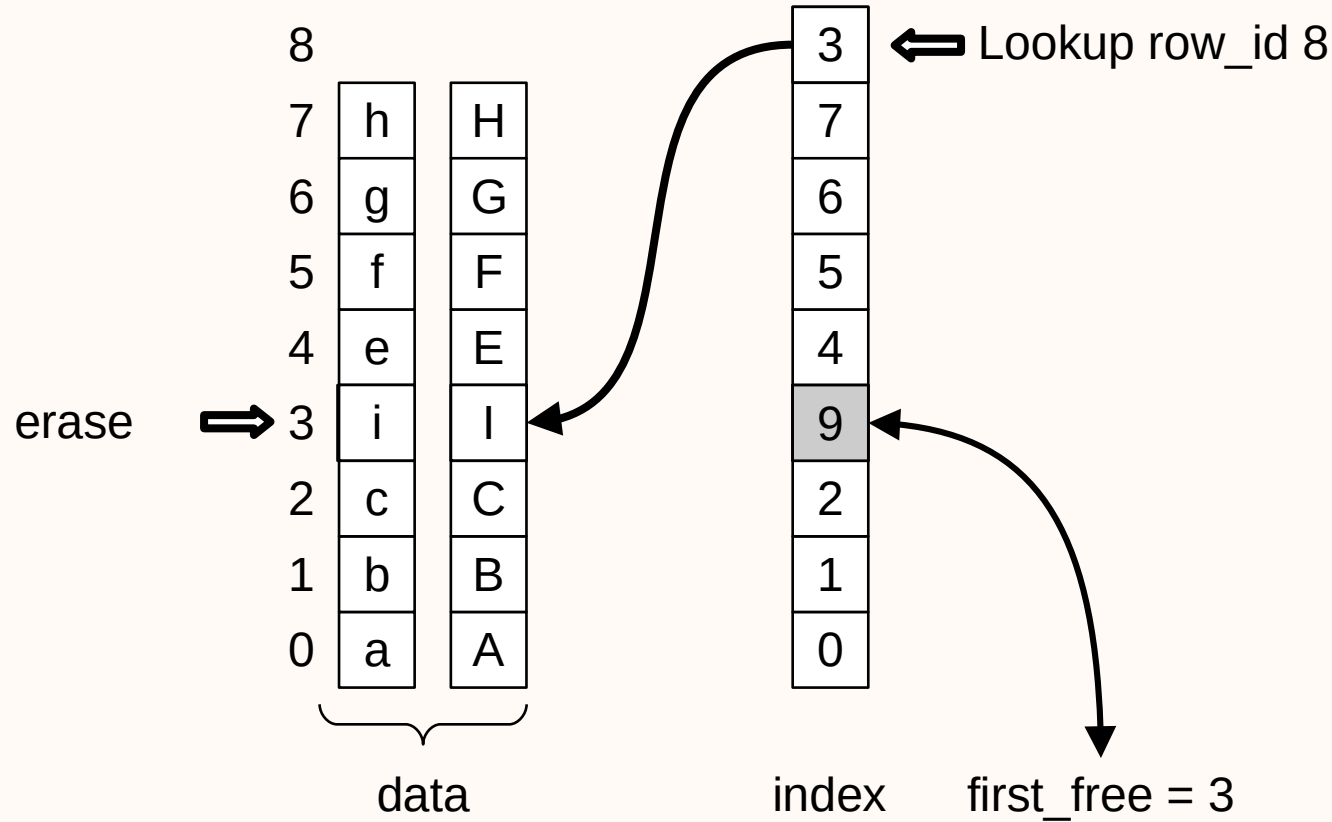
Stable row ID



Stable row ID



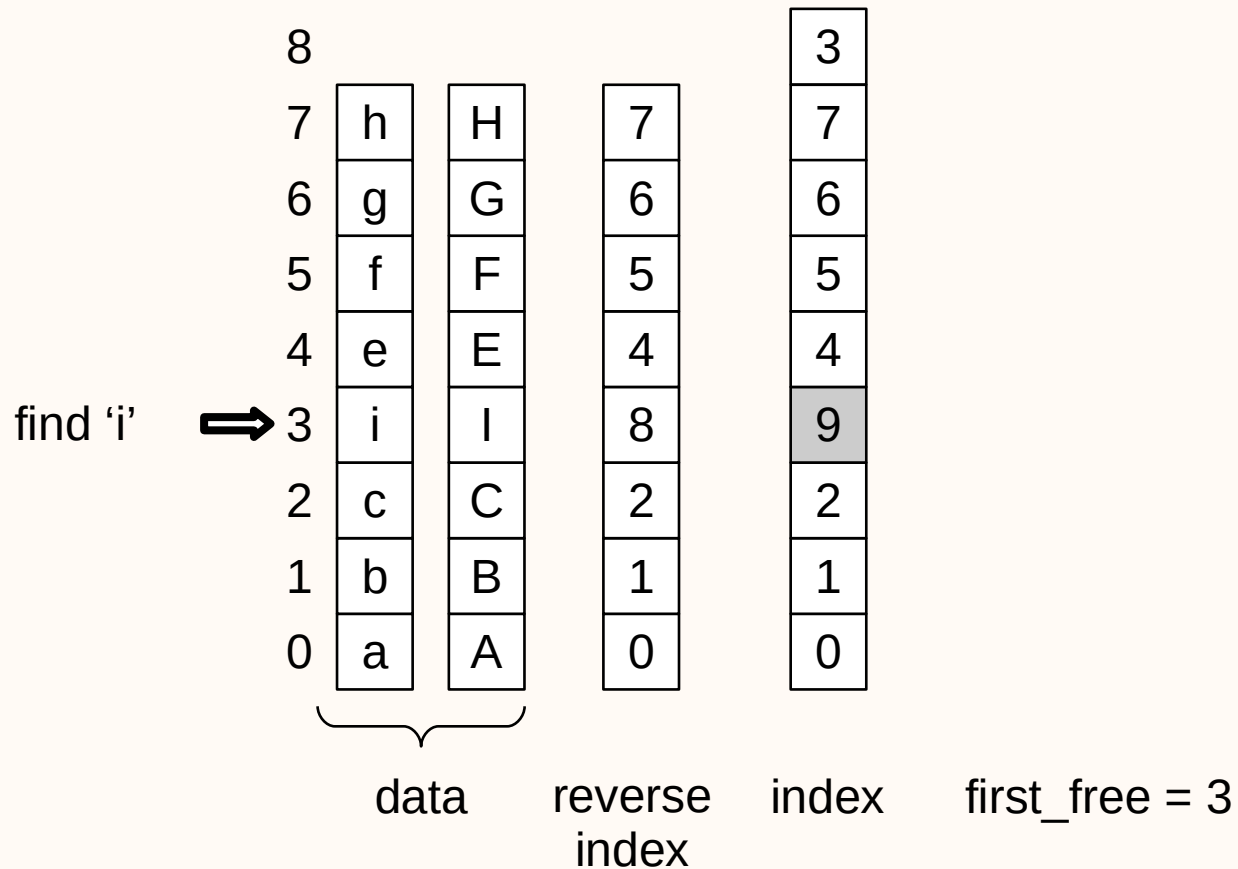
Stable row ID



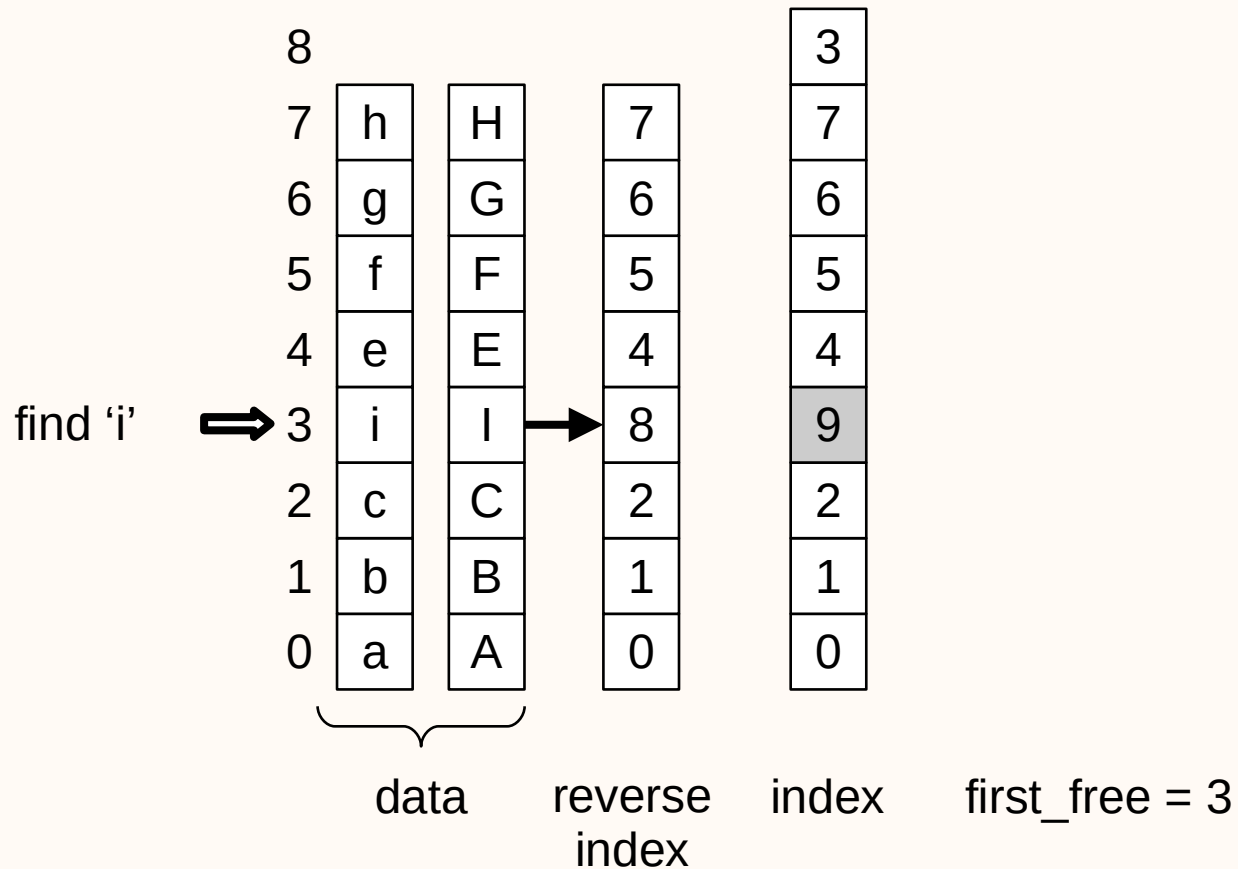
Stable row ID

8				3
7	h	H	7	7
6	g	G	6	6
5	f	F	5	5
4	e	E	4	4
3	i	I	8	9
2	c	C	2	2
1	b	B	1	1
0	a	A	0	0
	data		reverse index	index
				first_free = 3

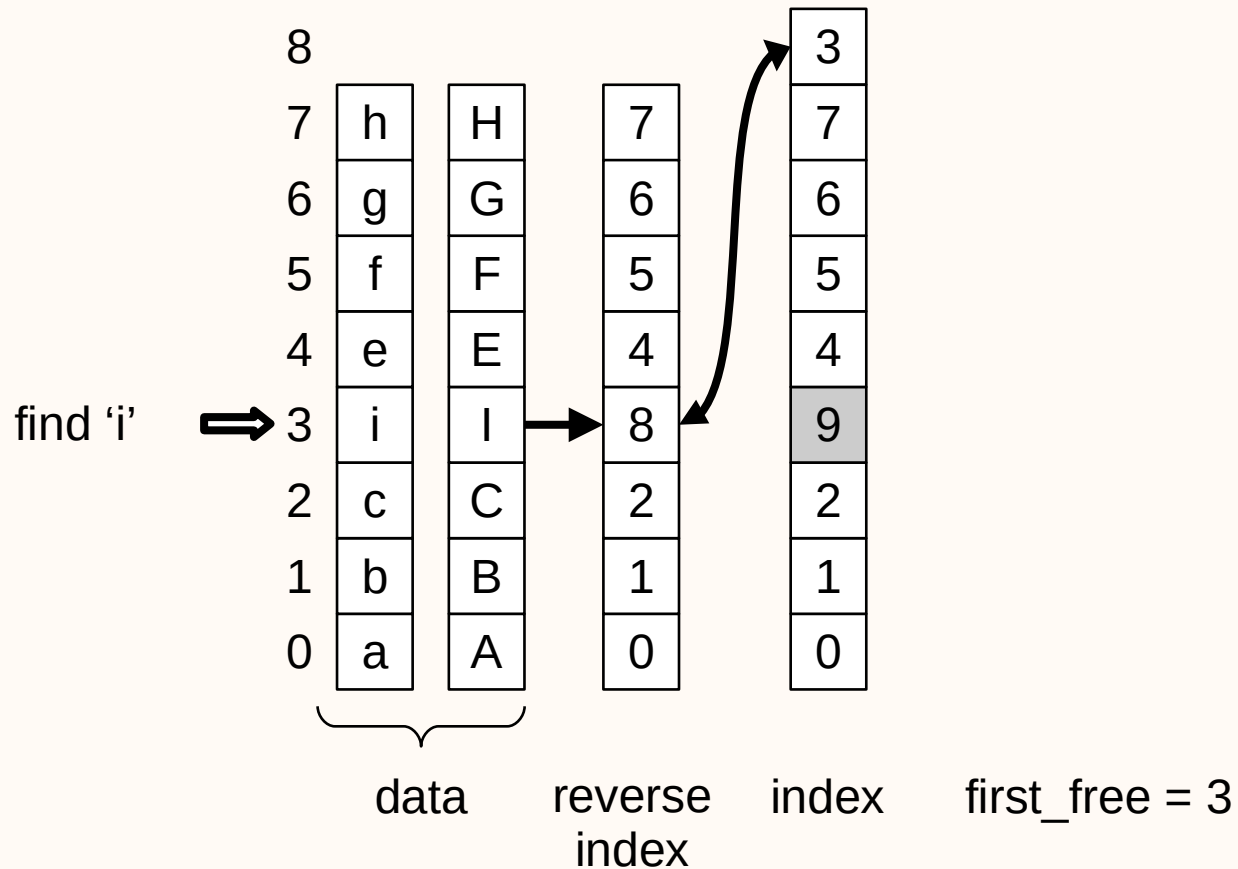
Stable row ID



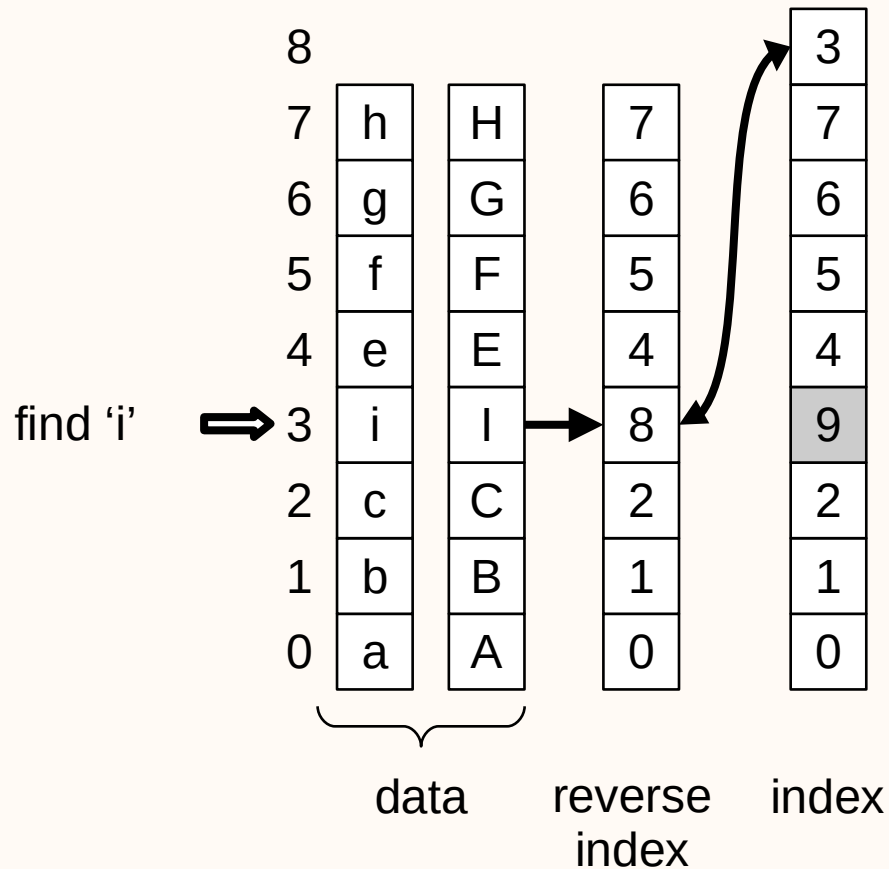
Stable row ID



Stable row ID



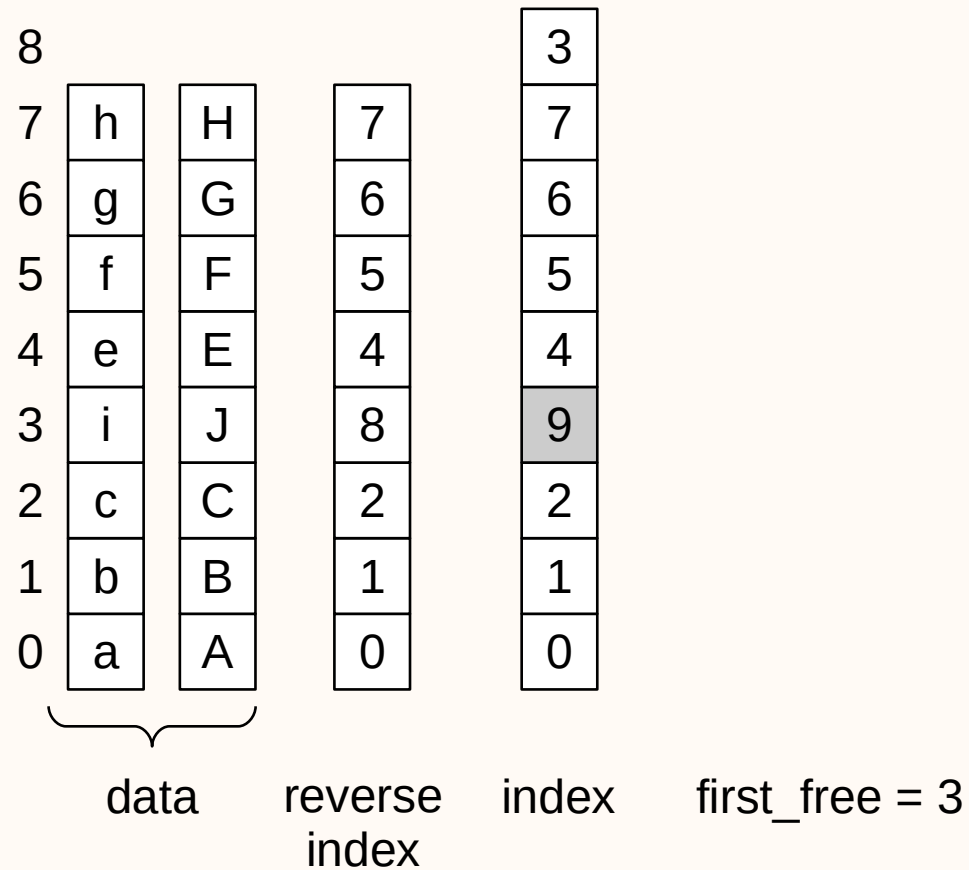
Stable row ID



'i' is at row_id 8,
which is stored at
offset 3 in the
columns.

first_free = 3

Stable row ID



Stable row ID

insert
'k', 'K'

8				3
7	h	H	7	7
6	g	G	6	6
5	f	F	5	5
4	e	E	4	4
3	i	J	8	9
2	c	C	2	2
1	b	B	1	1
0	a	A	0	0



data

reverse
index

index

first_free = 3

Stable row ID

insert
'k', 'K'

8				3
7	h	H	7	7
6	g	G	6	6
5	f	F	5	5
4	e	E	4	4
3	i	J	8	9
2	c	C	2	2
1	b	B	1	1
0	a	A	0	0
data			reverse	index
			index	

first_free = 3

Stable row ID

insert
'k', 'K'

8	k	K	3	3
7	h	H	7	7
6	g	G	6	6
5	f	F	5	5
4	e	E	4	4
3	i	J	8	9
2	c	C	2	2
1	b	B	1	1
0	a	A	0	0



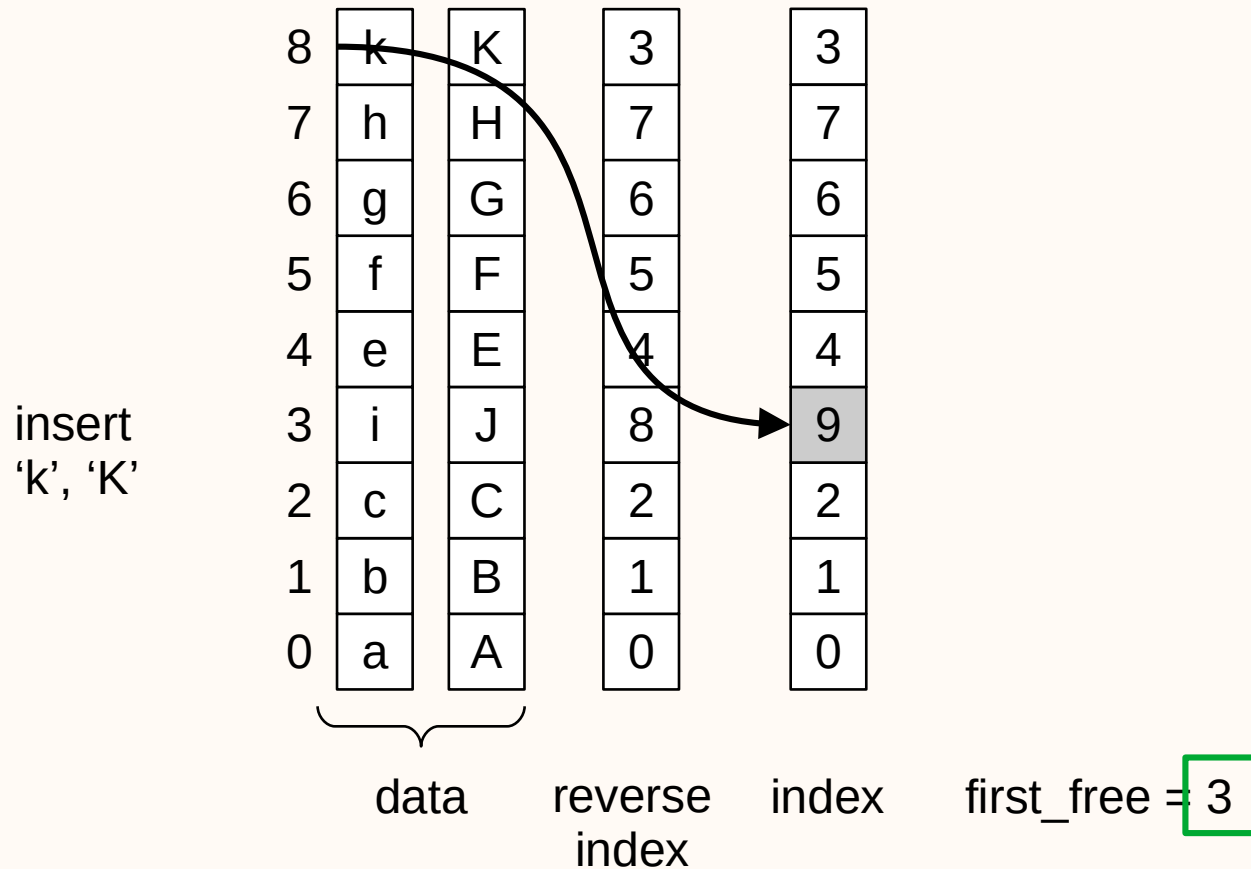
data

reverse
index

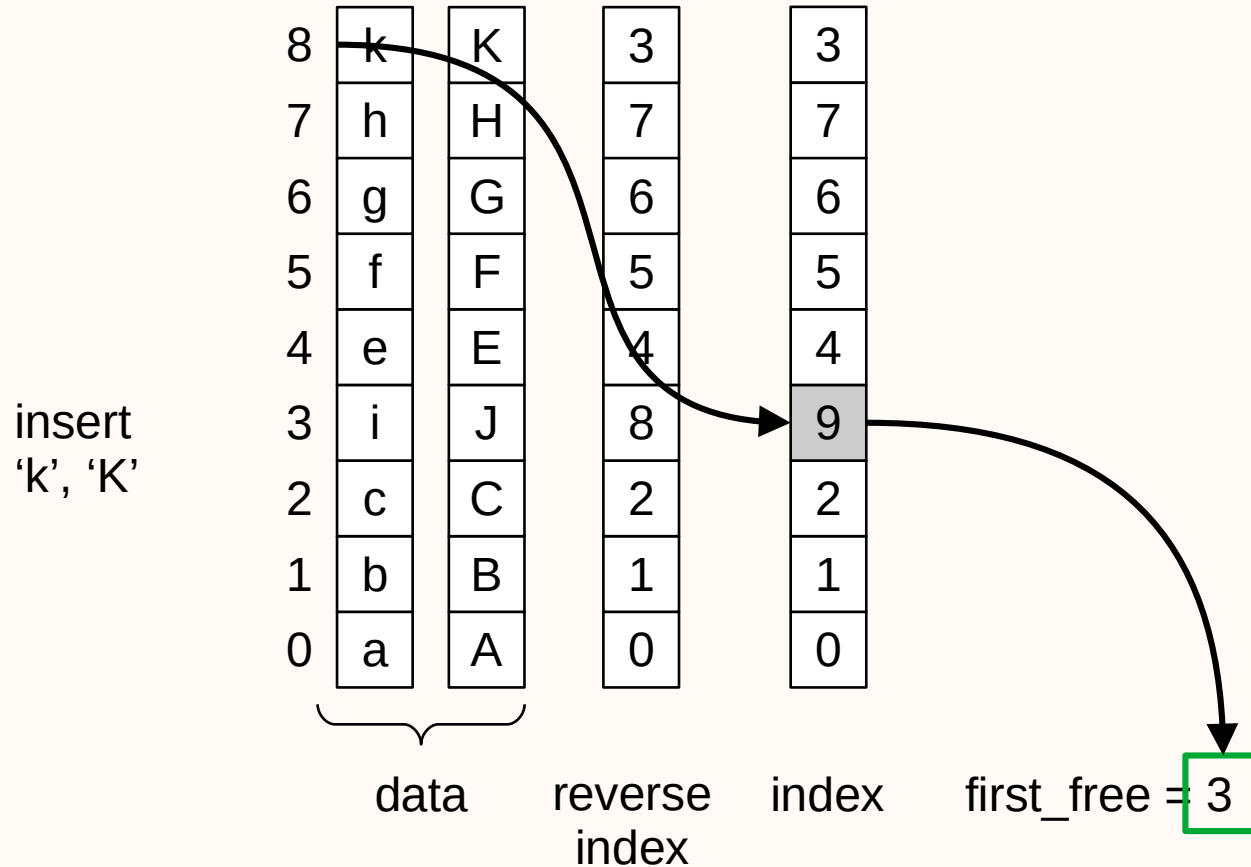
index

first_free = 3

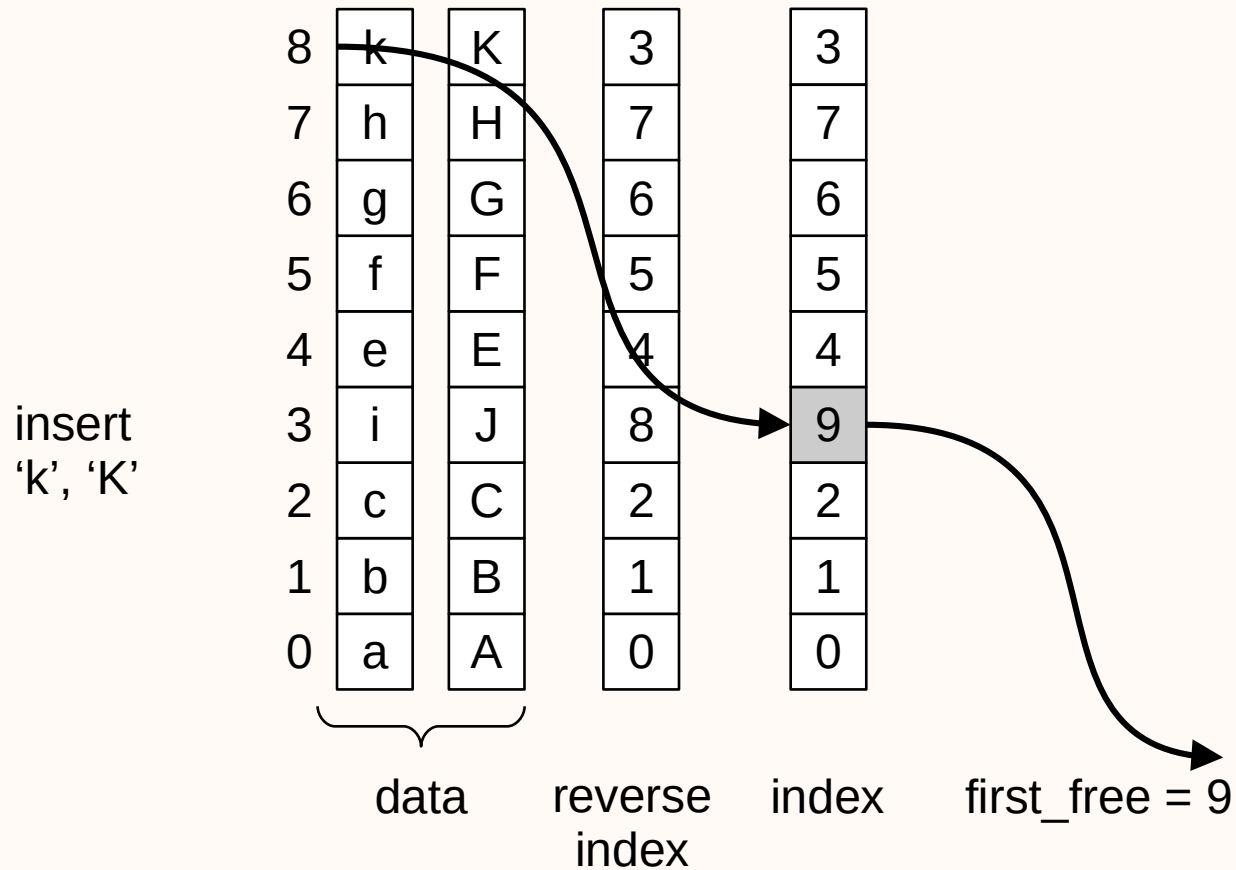
Stable row ID



Stable row ID



Stable row ID



Stable row ID

insert
'k', 'K'

8	k	K	3	3
7	h	H	7	7
6	g	G	6	6
5	f	F	5	5
4	e	E	4	4
3	i	J	8	8
2	c	C	2	2
1	b	B	1	1
0	a	A	0	0



data

reverse
index

index

first_free = 9

Stable row ID

insert
'k', 'K'

8	k	K	3	3
7	h	H	7	7
6	g	G	6	6
5	f	F	5	5
4	e	E	4	4
3	i	J	8	8
2	c	C	2	2
1	b	B	1	1
0	a	A	0	0



data

reverse
index

index

first_free = 9

Return row_id == 3,
which refers to the
new element

Stable row ID

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row operator[](row_id id) {
        auto offset = index_[id.offset].offset;
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>) {
            return row{std::get<Is>(data_)[offset]...};
        };
        return std::invoke(access, indexes);
    }
private:
    static constexpr auto indexes = std::index_sequence_for<Ts...>{};
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    row_id first_free_ = {0};
};
```

Stable row ID

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row operator[](row_id id) {
        auto offset = index[id.offset].offset;
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>) {
            return row{std::get<Is>(data_)[offset]...};
        };
        return std::invoke(access, indexes);
    }
private:
    static constexpr auto indexes = std::index_sequence_for<Ts...>{};
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    row_id first_free_ = {0};
};
```

Translate row_id to offset in the vectors

Stable row ID

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row operator[](row_id id) {
        auto offset = index[id.offset].offset;
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>) {
            return row{std::get<Is>(data_)[offset]...};
        };
        return std::invoke(access, indexes);
    }
private:
    static constexpr auto indexes = std::index_sequence_for<Ts...>{};
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    row_id first_free_ = {0};
};
```

The rest of the
access as before

Stable row ID

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row operator[](row_id id) {
        auto offset = index_[id.offset].offset;
        auto access = [&<size_t ... Is>(std::index_sequence<Is...>) {
            return row{std::get<Is>(data_)[offset]...};
        };
        return std::invoke(access, indexes);
    }
private:
    static constexpr auto indexes = std::index_sequence_for<Ts...>{};
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    row_id first_free_ = {0};
};
```

Stable row ID

```
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row_id push_back(Ts... ts);
    void pop_back();
    row operator[](row_id i);
    row back();
    void erase(row_id i);
    void reserve(size_t size);
    bool empty() const;
    size_t size() const;
    bool has_row(row_id i) const;

private:
    ...
};

...
};

... offset;
... Is>(std::index_sequence<Is...>) {
...>(data_)[offset]...};

... indexes);

... = std::index_sequence_for<Ts...>{};
... data_;
... index_;
```

Stable row ID

```
template <typename ... Ts>
class table {
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row_id push_back(Ts... ts);
    void pop_back();
    row operator[](row_id i);
    row back();
    void erase(row_id i);
    void reserve(size_t size);
    bool empty() const;
    size_t size() const;
    bool has_row(row_id i) const;

private:
    ...
};
```

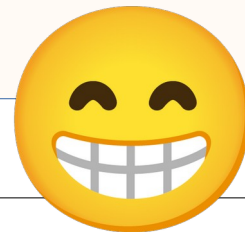
```
table<int,int,int,int> t;
auto i1 = t.push_back(1,2,3,4);
auto i2 = t.push_back(2,3,4,5);
auto i3 = t.push_back(3,4,5,6);
auto i4 = t.push_back(4,5,6,7);

t.erase(i2);
t.erase(i3);

assert(t[i1] == std::tuple(1,2,3,4));
assert(t[i4] == std::tuple(4,5,6,7));
assert(! t.has_row(i2));
assert(! t.has_row(i3));

data_,
index_;
```

Stable row ID



```
template <typename ... Ts>
class table {
template <typename ... Ts>
class table {
public:
    using row = std::tuple<Ts&...>;
    struct row_id { size_t offset; };

    row_id push_back(Ts... ts);
    void pop_back();
    row operator[](row_id i);
    row back();
    void erase(row_id i);
    void reserve(size_t size);
    bool empty() const;
    size_t size() const;
    bool has_row(row_id i) const;

private:
    ...
};
```

```
table<int,int,int,int> t;
auto i1 = t.push_back(1,2,3,4);
auto i2 = t.push_back(2,3,4,5);
auto i3 = t.push_back(3,4,5,6);
auto i4 = t.push_back(4,5,6,7);

t.erase(i2);
t.erase(i3);

assert(t[i1] == std::tuple(1,2,3,4));
assert(t[i4] == std::tuple(4,5,6,7));
assert(! t.has_row(i2));
assert(! t.has_row(i3));
```

Live Demo

row_id

Status

But, we also need...

... a stable row-ID to lookup objects

... an iterator interface to work with ranges

And want...

... a convenient way to look at only some columns

... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

And want...

- ... a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

And want...

- ... a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements



Iterators

- Iterating in storage order is fast!
 - The prefetcher loves predictable access patterns in contiguous memory
- Let the value type be a tuple of references to the elements of each column
 - Like `table<Ts...>::operator[]`
 - Compare with `ranges::zip_view`

Iterators

```
struct sentinel {};  
struct iterator {  
    using value_type = row;  
    using difference_type = ssize_t;  
  
    value_type operator*() const {  
        return std::invoke([&<size_t ... Is>(std::index_sequence<Is...>){  
            return value_type{std::get<Is>(t->data_)[offset]...};  
        },  
        t->indexes);  
    }  
    iterator& operator++() { ++offset; return *this; }  
    iterator operator++(int) { auto copy = *this; ++*this; return copy; }  
    bool operator==(const iterator&) const = default;  
    bool operator==(sentinel) const { return offset == t->size(); }  
    table* t;  
    ssize_t offset;  
};
```

Iterators

```

struct sentinel {};
struct iterator {
    using value_type = row;
    using difference_type = ssize_t;

    value_type operator*() const {
        return std::invoke([&<ssize_t ... Is>(std::index_sequence<Is...>){
            return value_type{std::get<Is>(t->data_)[offset]...};
        },
            t->indexes);
    }
    iterator& operator++() {
        iterator operator++(int) { return *this; return copy; }
    }
    bool operator==(const iterator&) const = default;
    bool operator==(sentinel) const { return offset == t->size(); }

    table* t;
    ssize_t offset;
};

```

Instead of one iterator
per column, use the table
and the column offset.

Iterators

```

struct sentinel {};
struct iterator {
    using value_type = row;
    using difference_type = ssize_t;

    value_type operator*() const {
        return std::invoke([&<size_t ... Is>(std::index_sequence<Is...>){
            return value_type{std::get<Is>(t->data_)[offset]...};
        },
            t->indexes);
    }

    iterator& operator++() { ++offset; return *this; }
    iterator operator++(int) { auto copy = *this; ++*this; return copy; }
    bool operator==(const iterator&) const = default;
    bool operator==(sentinel) const { return offset == 0; }
    table* t;
    ssize_t offset;
};

```

Dereferencing becomes
the same as the old
table::operator[]
on offset.

Iterators

```
struct sentinel {};
struct iterator {
```

Comparing with the *end-of-range* sentinel becomes checking if the *one-past-the-end* offset of the table has been reached.

```
    _t;

    {
        ze_t ... Is>(std::index_sequence<Is...>){
            ... std::get<Is>(t->data_)[offset]...};
    },
    t->indexes);
}
iterator& operator++() { ++offset; return *this; }
iterator operator++(int) { auto copy = *this; ++*this; return copy; }
bool operator--(const iterator&) const = default;
bool operator==(sentinel) const { return offset == t->size(); }
table* t;
size_t offset;
};
```

Iterators

```
template <typename ... Ts>
class table {
public:
    struct sentinel {};
    struct iterator { ... };
    friend class iterator;
    iterator begin() { return { this, 0 }; }
    sentinel end() const { return {}; }

    size_t size() const { return std::get<0>(data_).size(); }
private:
    static constexpr auto indexes = std::index_sequence_for<Ts...>{};
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    row_id first_free_ = {0};
};
```

Iterators

```
template <typename ... Ts>
class table {
public:
    struct sentinel {};
    struct iterator { ... };
    friend class iterator;

    iterator begin() { return { this, 0 }; }
    sentinel end() const { return {}; }

    size_t size() const { return std::get<0>(data_).size(); }
private:
    static constexpr auto indexes = std::index_sequence_for<Ts...>{};
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    row_id first_free_ = {0};
};
```

begin() and end() becomes trivially simple.

Live Demo

iterators

row_id

It would be nice to be able to get the `row_id` from `*iterator`, but how?

`std::tuple<types&...>` does not naturally lend itself to that.

row_id

It would be nice to be able to get the `row_id` from `*iterator`, but how?

`std::tuple<types&...>` does not naturally lend itself to that.

Inheritance as **`row : std::tuple<types&...> { ... }`**; could take care of it, but then we'd lose destructuring as **`auto [a,b,c] = *iterator;`** or **`auto [a,b,c] = table[id];`**

row_id

It would be nice to be able to get the `row_id` from `*iterator`, but how?

`std::tuple<types&...>` does not naturally lend itself to that.

Inheritance as **`row : std::tuple<types&...> { ... }`**; could take care of it, but then we'd lose destructuring as **`auto [a,b,c] = *iterator;`** or **`auto [a,b,c] = table[id];`**

The standard offers a way to implement destructuring for your own types by:

row_id

It would be nice to be able to get the `row_id` from `*iterator`, but how?

`std::tuple<types&...>` does not naturally lend itself to that.

Inheritance as **`row : std::tuple<types&...> { ... }`**; could take care of it, but then we'd lose destructuring as **`auto [a,b,c] = *iterator;`** or **`auto [a,b,c] = table[id];`**

The standard offers a way to implement destructuring for your own types by:
Specializing **`std::tuple_size<T>`**

row_id

It would be nice to be able to get the `row_id` from `*iterator`, but how?

`std::tuple<types&...>` does not naturally lend itself to that.

Inheritance as `row : std::tuple<types&...> { ... }`; could take care of it, but then we'd lose destructuring as `auto [a,b,c] = *iterator;` or `auto [a,b,c] = table[id];`

The standard offers a way to implement destructuring for your own types by:

Specializing `std::tuple_size<T>`

Specializing `std::tuple_element<idx, T>`

row_id

It would be nice to be able to get the `row_id` from `*iterator`, but how?

`std::tuple<types&...>` does not naturally lend itself to that.

Inheritance as `row : std::tuple<types&...> { ... }`; could take care of it, but then we'd lose destructuring as `auto [a,b,c] = *iterator;` or `auto [a,b,c] = table[id];`

The standard offers a way to implement destructuring for your own types by:

- Specializing `std::tuple_size<T>`

- Specializing `std::tuple_element<idx, T>`

- Implementing your own `get<Idx>(T&&)` or `T::get<Idx>()`

Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id { size_t offset; };
    row operator[](row_id id) { return { this, index_[id.offset].offset; } }
private:
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    ...
};

template <typename ... Ts> struct row;
template <typename ... Ts>
struct row<table<Ts...>> {
    template <size_t I>
    friend Ts...[I]& get(row& r) { return std::get<I>(r.t->data_)[offset]; }
    row_id id() const { return t->reverse_index_[offset]; }
    table* t;
    size_t offset;
};
```

Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id { size_t offset; };
    row operator[](row_id id) { return { this, index_[id.offset].offset; } }
private:
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    ...
};

template <typename ... Ts> struct row;
template <typename ... Ts>
struct row<table<Ts...>> {
    template <size_t I>
    friend Ts...[I]& get(row& r) { return std::get<I>(r.t->data_)[offset]; }
    row_id id() const { return t->reverse_index_[offset]; }
    table* t;
    size_t offset;
};
```

Which row in
the table

Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id { size_t offset; };
    row operator[](row_id id) { return { this, index_[id.offset].offset; } }
private:
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    ...
};

template <typename T>
template <typename ... Ts>
struct row<table<Ts...>> {
    template <size_t I>
    friend Ts...[I]& get(row& r) { return std::get<I>(r.t->data_)[offset]; }
    row_id id() const { return t->reverse_index_[offset]; }
    table* t;
    size_t offset;
};
```

Get the row offset by lookup in the index

Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id { size_t offset; };
    row operator[](row_id id) { return { this, index_[id.offset].offset; } }
private:
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    ...
};

template <typename ... Ts> struct row;
template <typename ... Ts>
struct row<table<Ts...>> {
    template <size_t I>
    friend Ts & get(row& r) { return std::get<I>(r.t->data_)[offset]; }
    row_id id() const { return t->reverse_index_[offset]; }
    table* t;
    size_t offset;
};
```

Getting the
row_id back
is trivial

Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id { size_t offset; };
    row operator[](row_id id) { return { this, index_[id.offset].offset; } }
private:
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index;
    std::vector<row_id> index_;
    ...
};

template <typename ... Ts> struct row;
template <typename ... Ts>
struct row<table<Ts...>> {
    template <size_t I>
    friend Ts...[I]& get(row& r) { return std::get<I>(r.t->data_)[offset]; }
    row_id id() const { return t->reverse_index[offset]; }
    table* t;
    size_t offset;
};
```

Reference the desired column with tuple lookup.

Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id { size_t offset; };
    row operator[](row_id id) { return { this, index_[id.offset].offset; } }
private:
    std::tuple<std::vector<Ts>...> data_;
    std::vector<row_id> reverse_index_;
    std::vector<row_id> index_;
    ...
};

template <typename ... Ts> struct row<table<Ts...>> {
template <size_t I>
friend Ts...[I]& get(row& r) { return std::get<I>(r.t->data_)[offset]; }
row_id id() const { return t->reverse_index_[offset]; }
table* t;
size_t offset;
};
```

C++26 parameter
pack indexing magic!



Our own row type

```
template <typename ... Ts>
class table {
public:
    struct row_id {
        row operator[](row_id) const;
    };
private:
    std::tuple<std::vector<row_id>, ...>
    std::vector<row_id>
    std::vector<row_id>
    ...
};

template <typename ... Ts> struct row;
template <typename ... Ts>
struct row<table<Ts...>> {
    template
    friend Ts
    row_id id(r) { return std::get<I>(r.t->data_)[offset]; }
    table* t;
    size_t offset;
};

template <size_t I, typename ... Ts>
struct std::tuple_element<I, row<Ts...> {
    using type = Ts...[I]&;
};

template <typename ... Ts>
struct std::tuple_size<row<Ts...>>
: std::integral_constant<size_t, sizeof...(Ts)>
{};
```

Opt in to
destructuring

Live Demo

iterators_row

Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ... an iterator interface to work with ranges

And want...

- ... a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

... a convenient way to look at only some columns

... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

... a convenient way to look at only some columns

... a convenient way to call functions with selected elements



Selecting columns

Selecting columns

With the current row type, we can select columns by explicitly using `get<I>(row)`.

Selecting columns

With the current row type, we can select columns by explicitly using `get<I>(row)`.

```
void func(const table<int, int, float, std::string>& data)
{
    for (auto r : data) {
        std::println("{} {}", get<0>(r), get<3>(r));
    }
}
```

Selecting columns

With the current row type, we can select columns by explicitly using `get<I>(row)`.

```
void func(const table<int, int, float, std::string>& data)
{
    for (auto r : data) {
        std::println("{} {}", get<0>(r), get<3>(r));
    }
}
```

Selecting columns

With the current row type, we can select columns by explicitly using `get<I>(row)`.

```
void func(const table<int, int, float, std::string>& data)
{
    for (auto r : data) {
        std::println("{} {}", get<0>(r) get<3>(r));
    }
}
```

Not exactly
convenient,
is it?



Different row type

```
template <typename ...>
struct table;

template <typename, typename> struct
row;

template <typename ... Ts, size_t ... Cs>
struct row<table<Ts...>, std::index_sequence<Cs...>> {
    using row_id = typename table<Ts...>::row_id;

    template <size_t I>
    friend auto& get(const row& r) {
        static constexpr std::array columns{Cs...};
        return std::get<columns[I]>(r.t->data_)[r.offset];
    }
    ...
    table<Ts...>* t;
    size_t offset;
};
```

Different row type

The columns in the table that are referenced by this row type

```
template <typename ...>
struct table;

template <typename, typename> struct
row;

template <typename ... Ts, size_t ... Cs>
struct row<table<Ts...>, std::index_sequence<Cs...>> {
    using row_id = typename table<Ts...>::row_id;

    template <size_t I>
    friend auto& get(const row& r) {
        static constexpr std::array columns{Cs...};
        return std::get<columns[I]>(r.t->data_)[r.offset];
    }
    ...
    table<Ts...>* t;
    size_t offset;
};
```

Different row type

```
template <typename ...>
struct table;

template <typename, typename> struct
row;

template <typename ... Ts, size_t ... Cs>
```

The columns in
the table that
are referenced
by this row type

```
row<table<int, char, double, long>, std::index_sequence<0,2,3>> r;
get<1>(r);
```

```
};
```

Different row type

```
template <typename ...>  
struct table;
```

```
template <typename, typename> struct  
row;
```

```
template <typename ... Ts, size_t ... Cs>
```

The columns in
the table that
are referenced
by this row type

```
row<table<int, char, double, long>, std::index_sequence<0,2,3>> r;  
get<1>(r);
```

```
};
```

Different row type

```
template <typename ...>  
struct table;
```

```
template <typename, typename> struct  
row;
```

```
template <typename ... Ts, size_t ... Cs>
```

The columns in
the table that
are referenced
by this row type

```
row<table<int, char, double, long>, std::index_sequence<0 2 3>> r;  
get<1>(r);
```

```
};
```

Different row type

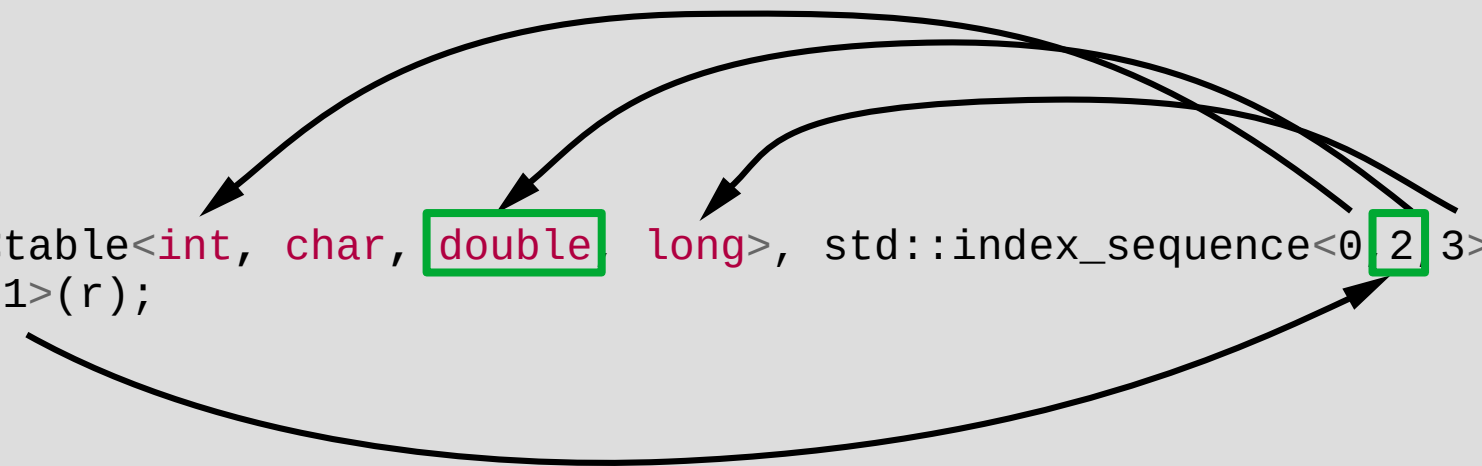
```
template <typename ...>  
struct table;
```

```
template <typename, typename> struct  
row;
```

```
template <typename ... Ts, size_t ... Cs>
```

The columns in
the table that
are referenced
by this row type

```
row<table<int, char, double long>, std::index_sequence<0 2 3>> r;  
get<1>(r);
```



```
};
```

Different row type

```
template <typename ...>
struct table;

template <typename, typename> struct
row;

template <typename ... Ts, size_t ... Cs>
struct row<table<Ts...>, std::index_sequence<Cs...>> {
    using row_id = typename table<Ts...>::row_id;

    template <size_t I>
    friend auto& get(const row& r) {
        static constexpr std::array columns{Cs...};
        return std::get<columns[I]>(r.t->data_)[r.offset];
    }
    ...
    table<Ts...>* t;
    size_t offset;
};
```

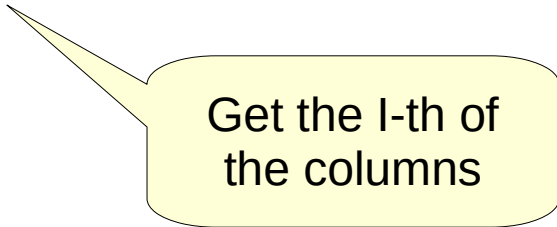
Different row type

```
template <typename ...>
struct table;

template <typename, typename> struct
row;

template <typename ... Ts, size_t ... Cs>
struct row<table<Ts...>, std::index_sequence<Cs...>> {
    using row_id = typename table<Ts...>::row_id;

    template <size_t I>
    friend auto& get(const row& r) {
        static constexpr std::array columns{Cs...};
        return std::get<columns[I]>(r.t->data_)[r.offset];
    }
    ...
    table<Ts...>* t;
    size_t offset;
};
```



Get the I-th of the columns

Different row type

```
template <typename Table, size_t ... Cs>
struct std::tuple_size<row<Table, std::index_sequence<Cs...>>>
: std::integral_constant<size_t, sizeof...(Cs)> {};

template <size_t I, typename ... Ts, size_t ... Cs>
struct std::tuple_element<I, row<table<Ts...>, std::index_sequence<Cs...>>>
{
    static constexpr std::array indexes = { Cs... };
    using type = Ts...[indexes[I]]&;
};
```

```
template <size_t I>
friend auto& get(const row& r) {
    static constexpr std::array columns{Cs...};
    return std::get<columns[I]>(r.t->data_)[r.offset];
}

...
table<Ts...>* t;
size_t offset;
};
```

Different row type

```
template <typename Table, size_t ... Cs>
struct std::tuple_size<row<Table, std::index_sequence<Cs...>>>
: std::integral_constant<size_t, sizeof...(Cs)> {};

template <size_t I, typename ... Ts, size_t ... Cs>
struct std::tuple_element<I, row<table<Ts...>, std::index_sequence<Cs...>>>
{
    static constexpr std::array indexes = { Cs... };
    using type = Ts...[indexes[I]]&;
};
```

The size is the number of columns referenced

```
template <size_t I>
friend auto& get(const row& r) {
    static constexpr std::array columns{Cs...};
    return std::get<columns[I]>(r.t->data_)[r.offset];
}

...
table<Ts...>* t;
size_t offset;
};
```

Different row type

```
template <typename Table, size_t ... Cs>
struct std::tuple_size<row<Table, std::index_sequence<Cs...>>>
: std::integral_constant<size_t, sizeof...(Cs)> {};

template <size_t I, typename ... Ts, size_t ... Cs>
struct std::tuple_element<I, row<table<Ts...>, std::index_sequence<Cs...>>>
{
    static constexpr std::array indexes = { Cs... };
    using type = Ts...[indexes[I]]&;
};
```

```
template <size_t I>
friend auto& get(const row& r, size_t I)
{
    static constexpr std::array indexes = { Cs... };
    return std::get<column<Ts...>, indexes[I].offset>(r);
}

...
table<Ts...>* t;
size_t offset;
};
```

The type of the I-th
element is the type
of the I-th column
referenced

Select columns in a row

```
template <size_t ... Is, typename Table, size_t ... Cs>
auto select(const row<Table, std::index_sequence<Cs...>>& r)
{
    static constexpr size_t columns[] { Cs... };
    return row<Table, std::index_sequence<columns[Is]...>>(r);
}
```

Select columns in a row

```
template <size_t ... Is, typename Table, size_t ... Cs>
auto select(const row<Table, std::index_sequence<Cs...>> & r)
{
    static constexpr size_t columns[] { Cs... };
    return row<Table, std::index_sequence<columns[Is]...>>(r);
}
```

Select a subset
of columns to
reference

Select columns in a row

```
template <size_t ... Is, typename Table, size_t ... Cs>
auto select(const row<Table, std::index_sequence<Cs...>>& r)
{
    static constexpr size_t columns[] { Cs... };
    return row<Table, std::index_sequence<columns[Is]...>>(r);
}
```

Return a row
that references
that subset

Select columns in a row

```
template <size_t ... Is, typename Table, size_t ... Cs>
auto select(const row<Table, std::index_sequence<Cs...>>& r)
{
    static constexpr size_t columns[] { Cs... };
    return row<Table, std::index_sequence<columns[Is]...>>(r);
}
```

```
drop_if(values, [](auto r) { auto [x,z] = select<0,2>(r); return x < z; });
```

Select columns in a row

```
template <size_t ... Is, typename Table, size_t ... Cs>
auto select(const row<Table, std::index_sequence<Cs...>>& r)
{
    static constexpr size_t columns[] { Cs... };
    return row<Table, std::index_sequence<columns[Is]...>>(r);
}
```

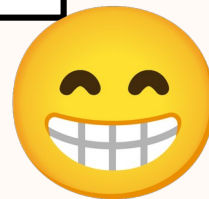
Select only the
columns we're
interested in for
the expression

```
drop_if(values, [](auto r) { auto [x,z] = select<0,2>(r); return x < z; });
```

Select columns in a row

```
template <size_t ... Is, typename Table, size_t ... Cs>
auto select(const row<Table, std::index_sequence<Cs...>>& r)
{
    static constexpr size_t columns[] { Cs... };
    return row<Table, std::index_sequence<columns[Is]...>>(r);
}
```

Select only the
columns we're
interested in for
the expression



```
drop_if(values, [](auto r) { auto [x,z] = select<0,2>(r); return x < z; });
```

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R&>().begin());
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs...>(*std::declval<r_iterator>()));

        iterator(const r_iterator& i) : r_iterator(i) {}
        auto operator*() const {
            const r_iterator& i = *this;
            return select<Cs...>(*i);
        }
    };
};

iterator begin() { return iterator{ r.begin() }; }
auto end() { return r.end(); }
R& r;
};
```

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R>().begin());
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs>(std::declval<r_iterator>()));

        iterator(const r_iterator& i) : r_iterator(i) {}
        auto operator*() const {
            const r_iterator& i = *this;
            return select<Cs...>(*i);
        }
    };

    iterator begin() { return iterator{ r.begin() }; }
    auto end() { return r.end(); }
    R& r;
};
```

range_selector refers to another range which has row<> as its value type.

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R&>().begin());
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs...>>(*std::declval<r_iterator>()));

        iterator(const r_iterator& i) : r_iterator(i) {}
        auto operator*() const {
            const r_iterator& i = *this;
            return select<Cs...>>(*i);
        }
    };
    iterator begin() { return iterator{ r.begin() }; }
    auto end() { return r.end(); }
    R& r;
};
```

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R&>().begin());
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs...>(r_iterator{}));

        iterator(const r_iterator& i) : r_iterator(i) {}
        auto operator*() const {
            const r_iterator& i = *this;
            return select<Cs...>(*i);
        }
    };
    iterator begin() { return iterator{ r.begin() }; }
    auto end() { return r.end(); }
    R& r;
};
```

Find the iterator type of the referenced range

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R>().begin());
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs...>(*std::declval<r_iterator>()));

        iterator(const iterator& i) : r_iterator(i) {}
        auto operator*(const iterator& i) const {
            const r_iterator r_i = i;
            return select<Cs...>(*r_i);
        }
    };
    iterator begin() { return iterator{ r.begin() }; }
    auto end() { return r.end(); }
    R& r;
};
```

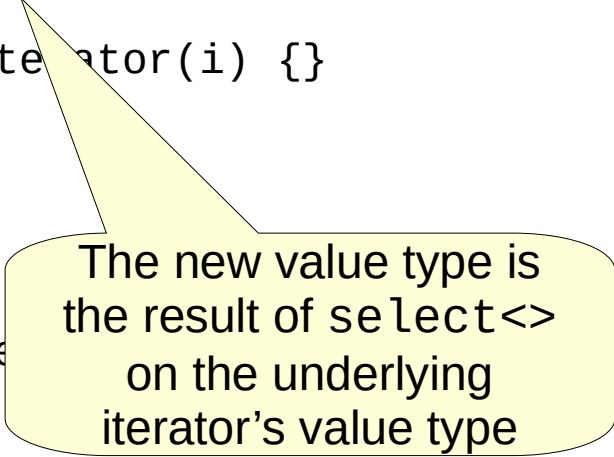
Create a new
iterator type via
inheritance

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R>().begin());
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs...>(*std::declval<r_iterator>()));

        iterator(const r_iterator& i) : r_iterator(i) {}
        auto operator*() const {
            const r_iterator& i = *this;
            return select<Cs...>(*i);
        }
    };
};

iterator begin() { return iterator{ r.begin() }; }
auto end() { return r.end(); }
R& r;
};
```



The new value type is the result of `select<>` on the underlying iterator's value type

Select columns in an iteration

```
template <typename R, size_t ... Cs>
struct range_selector {
    using r_iterator = decltype(std::declval<R>()).
    struct iterator : r_iterator {
        using difference_type = ssize_t;
        using value_type = decltype(select<Cs...>(*std::declval<r_iterator>()));

        iterator(const r_iterator& i) : r_iterator(i) {}
        auto operator*() const {
            const r_iterator& i = *this;
            return select<Cs...>(*i);
        }
    };
};

iterator begin() { return iterator{ r.begin() }; }
auto end() { return r.end(); }
R& r;
};
```

Return a row with the selected columns

Select columns in an iteration

```
template <size_t... Cs>
struct range_selector_maker {
    template <typename R>
    friend range_selector<R, Cs...> operator|(R& r, range_selector_maker)
    {
        return { r };
    }
};

template <size_t ... Cs>
range_selector_maker<Cs...> select() { return {}; }

};

iterator begin() { return iterator{ r.begin() }; }
auto end() { return r.end(); }
R& r;
};
```

Select columns in an iteration

```
template <size_t... Cs>
struct range_selector_maker {
    template <typename R>
    friend range_selector<R, Cs...> operator|(R& r, range_selector_maker)
    {
        return { r };
    }
};
```

Create a range_selector<>
by piping a range to the result
of select<>()

```
template <size_t ... Cs>
range_selector_maker<Cs...> select() { return {};
```

```
};
iterator begin() { return iterator{ r.begin() }; }
auto end() { return r.end(); }
R& r;
};
```

Select columns in an iteration

```
template <size_t... Cs>
struct range_selector_maker {
    template <typename R>
    friend range_selector<R, Cs...> operator|(R& r, range_selector_maker)
    {
        return { r };
    }
};
```

```
template <size_t ... Cs>
range_selector_maker<Cs...> select() { return {}; }
```

```
};
iterator
auto end(
    R& r;
};
```

```
for (auto [x,d] : values | select<0,3>()) {
    std::println("d={} x={}", d, x);
}
```

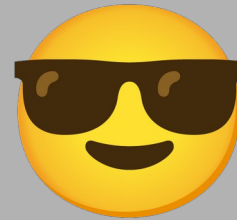
Select columns in an iteration

```
template <size_t... Cs>
struct range_selector_maker {
    template <typename R>
    friend range_selector<R, Cs...> operator|(R& r, range_selector_maker)
    {
        return { r };
    }
};
```

```
template <size_t ... Cs>
range_selector_maker<Cs...> select() { return {}; }
```

```
};
iterator
auto end(
R& r;
};
```

```
for (auto [x,d] : values | select<0,3>()) {
    std::println("d={} x={}", d, x);
}
```



Live Demo

row_select

Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

... a convenient way to look at only some columns

... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

- ✓ a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

- ✓ a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements



Apply functions to selected columns

```
drop_if(values, [](auto r) { auto [x,z] = select<0,2>(r); return x < z; });
```

Apply functions to selected columns

```
drop_if(values, [](auto r) { auto [x,z] = select<0,2>(r); return x < z; }));
```

```
template <size_t ... Cs, typename F>
auto select(F&& f)
{
    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)
    {
        return f(select<Cs...>(r));
    };
}
```

Apply functions to selected columns

drop_if(values,

Conflicts with

```
template <size_t ... Is, typename Table, size_t ... Cs>  
auto select(const row<Table, std::index_sequence<Cs...>>& r);
```

```
template <size_t ... Is, typename F>  
auto select(F&& f)  
{  
    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)  
    {  
        return f(select<Cs...>(r));  
    };  
}
```

Apply functions to selected columns

drop_if(values,

Conflicts with



```
template <size_t ... Is, typename Table, size_t ... Cs>  
auto select(const row<Table, std::index_sequence<Cs...>>& r);
```

```
template <size_t ... Is, typename F>  
auto select(F&& f)  
{  
    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)  
    {  
        return f(select<Cs...>(r));  
    };  
}
```

Apply functions to selected columns

drop_if(val

```
template <typename>
inline constexpr bool row_type_v = false;

template <typename T, typename Cs>
inline constexpr bool row_type_v<row<T,Cs>> = true;

template <typename T>
concept row_type = row_type_v<T>;
```

x < z; });

```
template <size_t ... Cs, typename F>
auto select(F&& f)
{
    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)
    {
        return f(select<Cs...>(r));
    };
}
```

Apply functions to selected columns

drop_if(val

```
template <typename>
inline constexpr bool row_type_v = false;

template <typename T, typename Cs>
inline constexpr bool row_type_v<row<T,Cs>> = true;

template <typename T>
concept row_type = row_type_v<T>;
```

x < z; });

```
template <size_t ... Cs, typename F>
auto select(F&& f)
{
    requires (! row_type<std::remove_cvref_t<F>>)

    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)
    {
        return f(select<Cs...>(r));
    };
}
```

Apply functions to selected columns

```
drop_if(values, [](auto r) { auto [x,z] = select<0,2>(r); return x < z; });
```

```
template <size_t ... Cs, typename F>
auto select(F&& f)
    requires (! row_type<std::remove_cvref_t<F>>)
{
    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)
    {
        return f(select<Cs...>(r));
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>([](auto r) { auto [x,z] = r; return x < z; }));
```

```
template <size_t ... Cs, typename F>
auto select(F&& f)
    requires (! row_type<std::remove_cvref_t<F>>)
{
    return [f = std::forward<F>(f)]<typename T, typename Is>(row<T, Is> r)
    {
        return f(select<Cs...>(r));
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>([](auto r) { auto [x,z] = r; return x < z; }));
```

```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)]<row_type R>(R r) {
        return std::invoke([&size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
            std::make_index_sequence<std::tuple_size_v<R>>{}));
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>([](auto r) { auto [x,z] = r; return x < z; }));
```

apply() takes a function and returns a lambda that takes a row.

```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)]<row_type R>(R r) {
        return std::invoke([&<size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
            std::make_index_sequence<std::tuple_size_v<R>>{}));
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>([](auto r) { auto [x,z] = r; return x < z; }));
```

```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)] row_type R>(R r) {
        return std::invoke([&size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
            std::make_index_sequence<std::tuple_size_v<R>>{}));
    };
}
```

When called, it calls the bound function `f` with every element in the row.

Apply functions to selected columns

```
drop_if(values, select<0,2>(apply([](auto x, auto z) { return x < z; })));
```

```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)]<row_type R>(R r) {
        return std::invoke([&<size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
            std::make_index_sequence<std::tuple_size_v<R>>{}));
    };
}
```

Apply functions to selected columns

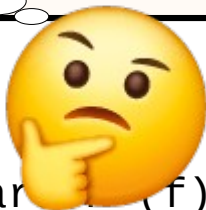
```
drop_if(values, select<0,2>(apply([](auto x, auto z) { return x < z; })));
```

```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)]<row_type R>(R r) {
        return std::invoke([&<size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
            std::make_index_sequence<std::tuple_size_v<R>>{}));
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>(apply([](auto x, auto z) { return x < z; })));
```

Isn't there a
name for this?



```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward(F)(f)]<row_type R>(R r) {
        return std::invoke([&<size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
            std::make_index_sequence<std::tuple_size_v<R>>{}));
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>(apply(std::less{})));
```

```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)]<row_type R>(R r) {
        return std::invoke([&]<size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
        std::make_index_sequence<std::tuple_size_v<R>>{});
    };
}
```

Apply functions to selected columns

```
drop_if(values, select<0,2>(apply(std::less{})));
```



```
template <typename F>
auto apply(F&& f)
{
    return [f = std::forward<F>(f)]<row_type R>(R r) {
        return std::invoke([&]<size_t ... Cs>(std::index_sequence<Cs...>) {
            return f(get<Cs>(r)...);
        },
        std::make_index_sequence<std::tuple_size_v<R>>{});
    };
}
```

Live Demo

function_select

Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

- ✓ a convenient way to look at only some columns
- ... a convenient way to call functions with selected elements



Status

But, we also need...

- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

- ✓ a convenient way to look at only some columns
- ✓ a convenient way to call functions with selected elements



Status

But, we also need...

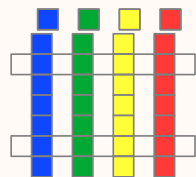
- ✓ a stable row-ID to lookup objects
- ✓ an iterator interface to work with ranges

And want...

- ✓ a convenient way to look at only some columns
- ✓ a convenient way to call functions with selected elements



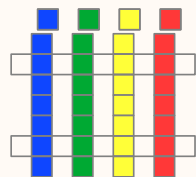
Cache Friendly + Functional + Ranges = ❤️



<https://github.com/rollbear/columnist>

Give it a go, and please help out if you can

Cache Friendly + Functional + Ranges = ❤️



<https://github.com/rollbear/columnist>

Give it a go, and please help out if you can

Source examples from this presentation
https://github.com/rollbear/cache_functional_ranges

Cache Friendly + Functional + Ranges = ❤️

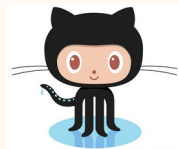
Björn Fahlner



bjorn@fahller.se



[@bjorn_fahller@mastodon.social](https://mastodon.social/@bjorn_fahller)



#include <C++>



[@rollbear](https://rollbear.com)